



Research paper

Grinding mill optimisation using grind curves and continuum-armed bandits

Jacques Olivier, William John Shipman*

Measurement and Control Division, Mintek, Private Bag X3015, Randburg, 2125, Gauteng, South Africa



ARTICLE INFO

Keywords:

Grinding mills
Real-time optimisation
Stochastic optimisation
Reinforcement learning
Continuum-armed bandit
Process control

ABSTRACT

Grinding mills exhibit steady-state behaviour, formalised by quadratic performance functions called grind curves. Grind curves are predominantly affected by feed-ore characteristics that are generally unknown during regular operation. This work assumes that optimising grinding mills, using grind curves, is akin to bandit problems that can be solved using continuum-armed bandit algorithms. This paper proposes a constrained continuum-armed bandit agent based on Thompson sampling that uses linear models to model the agent reward as a combination of real-value and binary components. Real-value reward components represent mill-performance indicators and are modelled as second-order polynomials of the agent's actions, thereby approximating the unknown grind curves. Binary components indicate constraint violations in the environment and are used to train linear constraint models for the optimisation algorithm selecting the optimal actions. Agent testing occurred in an environment containing a dynamic grinding mill model calibrated to grind curve data. Optimisation performance was compared to extremum-seeking controllers operating in the same environment. The experiments demonstrate that the continuum-armed bandit agent could find the unknown grind curves and steer the process near the optimal operating point. The continuum-armed bandit agent followed similar trajectories to the extremum-seeking controllers in most experiments. The agent requires tuning a smaller number of hyper-parameters than the extremum-seeking controllers, with physically meaningful interpretations. The agent improved upon the optimisation performance of one variant of the extremum-seeking controllers by 2% while producing a 1% decrease compared to the remaining two controllers. The agent proved robust to algorithm hyper-parameters, obtaining similar rewards for various agent configurations.

1. Introduction

Grinding mills are responsible for the size reduction of ores in mineral-processing plants. Grinding mills consist of large-diameter rotating drums, often loaded with additional steel balls. The rotation of the drum imparts energy to the mill contents (charge), and the resultant tumbling action of the charge causes breakage. Adequate size reduction is critical to ensure sufficient liberation of valuable minerals, generally embedded in a matrix of waste rock, for downstream separation processes.

Grinding circuits are renowned for being prone to frequent disturbances and significant process drift. Since much of the grinding in mills occurs autogenously (self-grinding), mill operation is sensitive to fluctuations in competency and the particle size of feed ores. These uncertainties, coupled with strongly interacting variables and non-linear performance functions (Powell et al., 2009), usually necessitate advanced process control (APC) systems for mill control. Among APC methods, model-predictive controllers are the most prevalent in mineral processing plants (Olivier and Craig, 2017). Implementing

such strategies is often constrained by the lack of direct measurement of the mill charge (Góralczyk et al., 2020) and fundamental process models. Consequently, many grinding mills are still operated with proportional-integral-derivative (PID) controllers. Optimal mill operation and control remains an active research area (Craig, 2012; Napier-Munn, 2015).

The composition of the mill charge is the primary driver of mill performance. More specifically, the density, particle-size distribution, and volumetric filling affect the shape of the charge, strongly affecting the product's particle-size distribution, production rates and power demand (Morrell (2003), Owen and Cleary (2015), Cleary and Owen (2019)). In practice, only the mill weight (load) is measured externally and related to the volumetric filling through internal inspections during mill shutdowns (Van der Westhuizen and Powell, 2006).

Mill-performance indicators generally include the fine-ore production rate (throughput), mill power draw, and the product's particle size distribution (grind). The estimated filling level is commonly related to each performance indicator using grind curves (Powell and Mainza,

* Corresponding author.

E-mail addresses: jacqueso@mintek.co.za (J. Olivier), williams@mintek.co.za (W.J. Shipman).

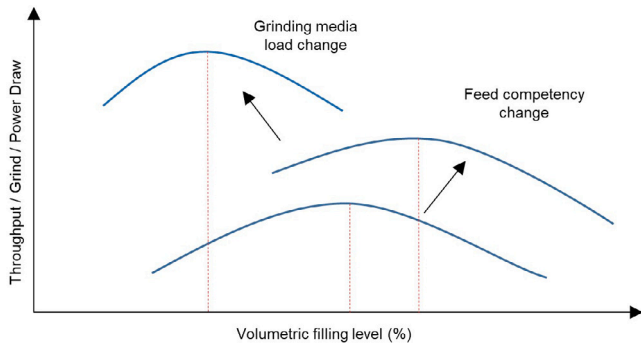


Fig. 1. Generic grind curve for a grinding-mill performance indicator. Dashed lines indicate the optimum filling level in each scenario.

2006; Powell et al., 2009). Grind curves represent each performance indicator's response to a changing fill level and mill speed. Grind curves exhibit a characteristic quadratic shape, as shown for the volumetric filling level in Fig. 1.

Grind curves define a surface upon which the mill operates when at a steady state. The quadratic shape allows for optimising a performance indicator by operating at the mill filling and speed found at the peak. As depicted in Fig. 1, grind curves are affected by the feed-ore characteristics, the load of steel balls (grinding media) and the state of the mill's internal steel liners. Aside from the feed-ore characteristics, these variables change over significantly longer timescales than the operational variables used for real-time control or the hours-days time horizons generally considered for real-time optimisation. The internal mill liners gradually wear over approximately six months before being replaced (McBride and Powell, 2006). Grinding media consumption rates are generally estimated with empirical models (Radziszewski and Martins, 2019), and ball-recharge rates are modified to maintain steady grinding-media loads. Furthermore, Powell et al. (2009) demonstrated that these variables do not change the quadratic shape of the underlying grind curve. Consequently, the effects of these variables are expressed in a measured grind curve, but the variables could be considered to remain constant in the time frames considered for optimisation.

Under these assumptions, the observed grind-curve relationships become characteristic of the ore types or blends fed to the mill. Practitioners can generate grind curves for all known ore sources on a plant using the method proposed by Powell et al. (2009) or Hilden et al. (2021). However, generating a grind curve for each ore type is generally infeasible since this would require extensive manual sampling campaigns and numerous mill shutdowns (Powell et al., 2009). Manual measurements cannot account for liner degradation or longer-term drift in the grinding media loading. Alternatively, one can leverage prior knowledge of the quadratic shape of grind curves and use curve-fitting methods to parameterise the quadratic function using process data. Du Preez et al. (2006) reported on such an optimiser that incrementally steps the load to estimate the power-load grind curve. Expert system rules codify similar logic to maintain the grinding process near the optimum (Borell et al., 1996; Hadzadeh et al., 2017; Bradford et al., 2021). However, these approaches could be more prevalent in practice based on the value of grind-curve data for optimisation.

Ziolkowski et al. (2022) recently investigated the use of grind curves as an objective function for extremum-seeking control (ESC) of grinding mills. ESC is a model-free, adaptive control method that systematically traverses an unknown map between inputs and an objective function in search of an optimum (Krstic and Wang, 2000). Ziolkowski et al. (2022) simulated a grinding mill using a non-linear dynamic mill model calibrated to published grind-curve data by Le Roux et al. (2020). The investigation demonstrated that ESCs could successfully locate the peak of the unknown grind curves and steer the simulated process to the vicinity of the theoretical optimum.

Extremum-seeking methods generally perturb the system to estimate gradients for optimisation. Alternatively, a reinforcement-learning agent could search for the optimal policy for optimisation through direct interaction with the process. Reinforcement learning (RL) is a class of learning algorithms that involves training a model, or agent, to learn a policy to achieve a goal through interaction with an environment (Sutton and Barto, 2018). During each step, the agent receives a reward quantifying the results of its previous action. Through trial and error, the agent attempts to maximise the reward gained by learning a policy, often without explicit knowledge of the system's dynamics. More recent progress in deep-function approximation has allowed for the successful extension of RL algorithms to high-dimensional observation spaces for discrete (Mnih et al., 2015) and continuous action spaces (Lillicrap et al., 2016). The adaptive nature of RL algorithms has generated further interest in their use in optimal control (Nian et al., 2020).

Many RL applications in the mining and mineral-processing industries have focused on planning and scheduling optimisation. Recent examples include short-term production scheduling in mining complexes (Kumar et al., 2020; Levinson et al., 2023), fleet dispatching for haulage vehicles (Huo et al., 2023), and ore blending (Feng et al., 2023). Fewer applications of RL are available in the supervisory and optimal control space, especially in mineral processing. Hallen et al. (2019) and Guo et al. (2019) investigated RL for grinding-circuit control. Hallen et al. (2019) studied the use of the Proximal Policy Optimisation (PPO) (Schulman et al., 2017) algorithm for regulatory controllers in a simulated milling circuit. Guo et al. (2019) used an actor-critic architecture to train a supervisory controller for a ball-milling circuit. Both controllers demonstrated the ability to increase mill throughput, subject to certain physical and operational constraints.

This work assumes that optimising grinding mills is akin to solving a bandit problem. Bandit problems involve selecting an optimal decision from a set of possible choices where the reward of each option is stochastic and unknown. Likewise, the optimal operating point in a grinding mill depends on the unknown characteristics of the incoming ore rather than the state of the mill. Bandit algorithms describe a process whereby an agent explores the possible set of actions, or action space, and learns the optimal choice of action or policy. The agent typically learns the optimal policy by modelling the reward as a function of the agent's actions.

Bandit algorithms are specialised for decision-making in a nonassociative setting, in that decisions are made independent of the agent's state (Sutton and Barto, 2018, Chapter 2). At each time step t , the agent takes an action A_t and receives a reward R_t from the environment. An arbitrary action a has an expected reward called the action value, defined by the q -value or action-value function in Eq. (1).

$$q_*(a) = \mathbb{E}[R_t | A_t = a] \quad (1)$$

This work's action-value functions represent the grind curves approximated as second-order polynomials of real-value reward components. The action values are initially unknown, and action-value estimates are iteratively refined as the bandit algorithm explores the action space. If the action space is discrete, where $a_i \in \{a \mid a_i \in \mathbb{Z}^{2 \leq i \leq k}\}$, the problem is called a k -armed bandit problem. The continuum-armed bandit (CAB) is an extension of the k -armed bandit to a continuous action space (Agrawal, 1995), where actions are represented as a vector. CAB algorithms typically learn to model the reward as a linear function of the actions taken (Agrawal and Goyal, 2012b; Abeille, 2017, Chapter 2; Agarwal et al., 2020; Lattimore and Szepesvári, 2020).

This work presents the optimisation and control of a simulated grinding mill using a CAB algorithm. The agent was adapted from Shipman (2021), which presented preliminary testing in a steady-state environment. In this work, the agent underwent rigorous testing in a dynamic simulation environment, and critical hyper-parameters for operationalising the agent were identified.

The CAB agent models the reward as an arbitrary combination of real-value and binary components. Real-value reward components, such as mill-performance indicators, are measured and modelled as polynomials of known order using robust linear regression. Binary reward components indicate constraint violations in the environment. Binary rewards train logistic regression classifiers used in the optimisation algorithm for action selection. The agent effectively decomposes a complex reward function into several smaller functions that can be approximated using linear models. The agent is from the probability-matching class of bandit algorithms and uses Thompson sampling (Thompson, 1933) for exploration (Amin et al., 2021). These algorithms select actions according to their probabilities of being the optimal actions. Gaussian distributions are formed over the parameter vectors of the regression models and sampled for candidate grind curves. The agent was implemented in an environment containing a non-linear dynamic model (Le Roux et al., 2020) to repeat selected experiments presented in Ziolkowski et al. (2022) with the CAB optimiser. The agent is a real-time optimiser, providing setpoints to lower-level regulatory controllers. Throughput optimisation and weighted throughput-grind optimisation experiments performed by Ziolkowski et al. (2022) are repeated in this work. The three ESCs employed in their paper served as a baseline for the performance evaluation of the CAB agent.

The main contribution of this work was the testing and validation of a novel real-time optimiser for grinding mills. The level of uncertainty in process measurements was used to guide the optimiser's exploration. The optimiser can handle multiple constraints and dynamic components in the reward function. Furthermore, two hyper-parameters critical for the industrial implementation of the optimiser were identified, and their effects on the optimiser's performance were evaluated. The remainder of the paper is structured as follows: Section 2 describes the analytical methodology, the simulation environment and the CAB algorithm. Section 3 demonstrates the application of the bandit algorithm in three different optimisation tasks, and Section 4 briefly discusses the results. Section 5 summarises the conclusions.

2. Methodology

This section describes the analytical methodology applied for grinding-mill optimisation. The first subsection provides a brief overview of agent–environment interactions. The remaining subsections provide details of the environment and agent.

2.1. Overview

Figs. 2 and 3 collectively portray the agent–environment interaction. Fig. 2 portrays the agent–environment interaction during a throughput optimisation experiment with N_b arbitrary constraints added.

Agent–environment interactions are briefly outlined below to aid in the following discussion. The step numbers correspond to those given in Fig. 2. The procedure is formalised algorithmically at the end of this section.

1. At time t , the environment receives the action $\mathbf{A}_t$ from the agent, containing setpoints for the mill filling and mill speed.
2. The environment, consisting of the simulation plant in Fig. 3, is allowed to loop for N_{sf} time periods for the proportional–integral (PI) controllers to steer the process towards the new setpoints. N_{sf} denotes the agent's step frequency, considered a hyper-parameter to the CAB algorithm.
3. The environment builds a reward vector, $\mathbf{R}_{t+N_{sf}}$, according to Eq. (2) from the final measurements calculated at time $t + N_{sf}$. In Eq. (2), N_R denotes the number of real-value reward components, while N_b denotes the number of binary reward components. The composition of $\mathbf{R}_{t+N_{sf}}$ is determined by the

optimisation objective. The reward vector is transferred to the agent:

$$\mathbf{R}_{t+N_{sf}} = \langle \mathbf{R}_{j,t+N_{sf}} \rangle; \quad j \in \mathbb{Z}^{1 \leq j \leq N_R + N_b} \quad (2)$$

where $\mathbf{R}_{j,t+N_{sf}} \in \mathbb{R}$ if $j \in \mathbb{Z}^{1 \leq j \leq N_R}$ and $\mathbf{R}_{j,t+N_{sf}} \in \{0,1\}$ if $j \in \mathbb{Z}^{N_R < j \leq N_R + N_b}$.

4. The agent stores the sample vector of $\langle \mathbf{A}_t, \mathbf{R}_{t+N_{sf}} \rangle$ into $N_R + 2N_b$ replay buffers of experiences $\mathcal{D}_j \in \mathbb{R}^{100 \times N}$, with a maximum of 100 samples. N is the number of actions, representing the dimensionality of $\mathbf{A}_t$. As explained in Section 2.3.4, each of the N_b binary reward components is associated with two replay buffers containing up to 100 samples each. One replay buffer stores samples that demonstrate constraint violation, while the other replay buffer stores samples where that constraint was not violated, leading to $2N_b$ replay buffers in total for the binary reward components.
5. The agent uses the set of experiences in $\mathcal{D}$ to train linear models $l_j \in \mathcal{L}, j \in \mathbb{Z}^{1 \leq j \leq N_R}$, with parameter vectors $\theta_j \in [\theta_j]$ mapping past actions to each of the N_R real-value reward components using robust linear regression. $[\theta_j]$ represents the matrix of model parameter vectors for the regression models in list $\mathcal{L}$.
6. The agent uses the set of experiences in $\mathcal{D}$ to train logistic regression models $c_j \in \mathcal{C}, j \in \mathbb{Z}^{1 \leq j \leq N_b}$, with parameter vectors $\mathbf{w}_j \in [\mathbf{w}_j]$ for each of the N_b binary reward components in $\mathbf{R}_{t+N_{sf}}$ to function as constraints in the action space. $[\mathbf{w}_j]$ represents the matrix of model parameter vectors for the classification models in list $\mathcal{C}$.
7. The agent uses Thompson sampling to explore the action space. The agent generates a Gaussian distribution over the parameter vectors $\theta_j \in [\theta_j]$, for each real-value reward component and samples candidate parameter vectors $\hat{\theta}_j \in [\hat{\theta}_j]$, from each of the distributions.
8. The agent aggregates the real-value reward models $\mathcal{L}([\hat{\theta}_j])$, into a single scalar reward function, $\mathcal{R}_{t+N_{sf}}$, according to the objective function $\lambda(\mathcal{L})$. This step is not shown in Fig. 2 but is demonstrated in the experiments.
9. The reward function $\mathcal{R}_{t+N_{sf}}$ is maximised subject to constraints imposed by the set of N_b constraint models $\mathcal{C}$ and limits between successive agent steps supplied as a hyper-parameter δ . The mill filling and speed at the maximum are concatenated into $\mathbf{A}_t$ and sent to the environment.

Detailed descriptions of each of the components are provided in the subsections below.

2.2. Environment description

The environment is a simulation of a typical grinding mill, depicted as a block diagram in Fig. 3. The environment is identical to the model described by Ziolkowski et al. (2022). It comprises a dynamic, non-linear grinding-mill model with two regulatory proportional–integral (PI) controllers. The population-balance model describes the movement of particles discretised into broad classes based on particle size. The grinding-mill model forms part of a larger grinding-circuit model previously used in controller testing and development (Coetzee et al., 2010; Le Roux et al., 2013, 2016).

The model was implemented in Python and wrapped into a simulation environment compatible with the OpenAI Gym interface (Brockman et al., 2016). For compatibility with the results of Ziolkowski et al. (2022), the density set point indicated in Fig. 3 remained at a constant value of 0.9.

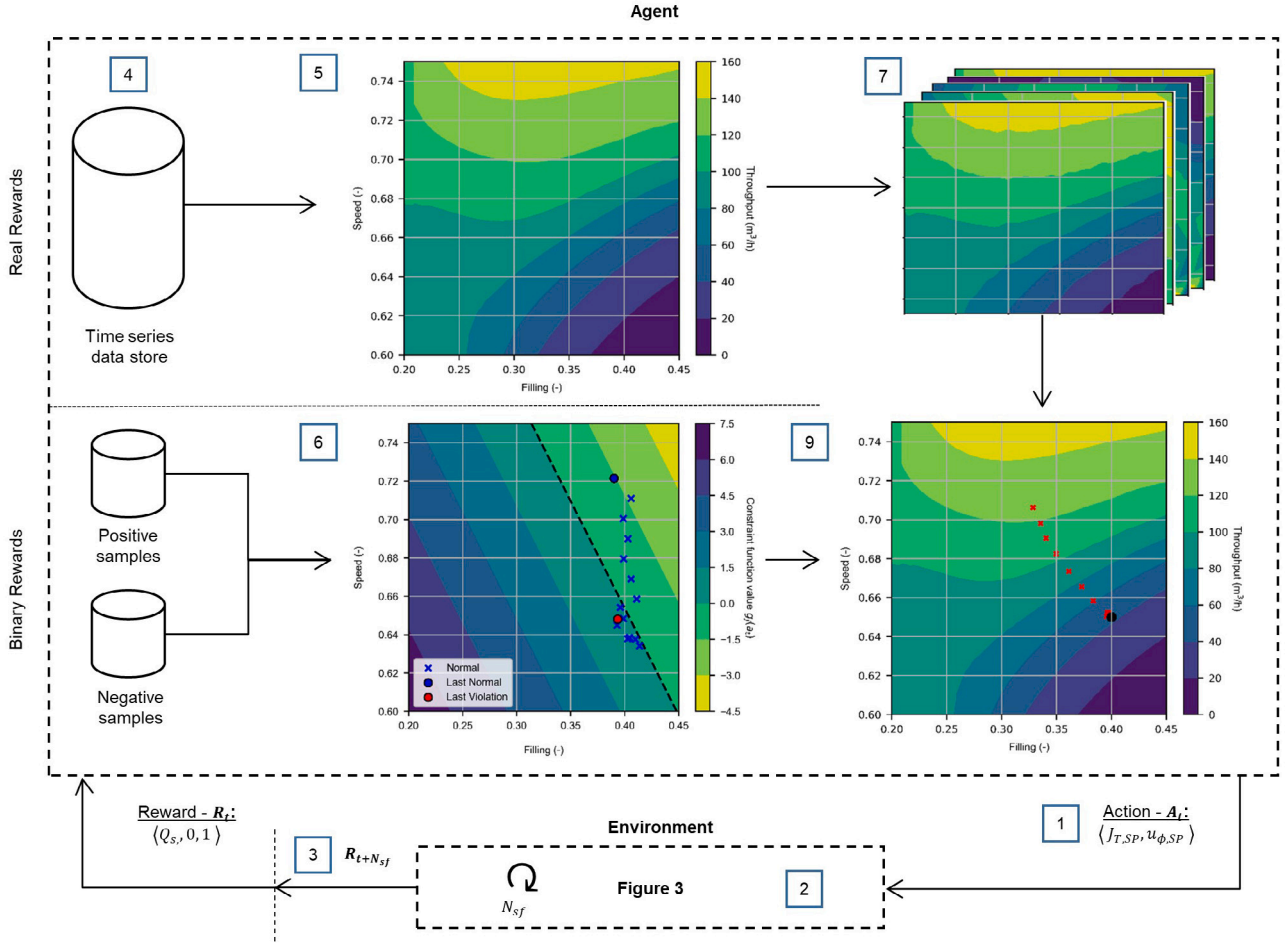


Fig. 2. Agent-environment interaction for an experiment requiring mill throughput as a real-value reward component.

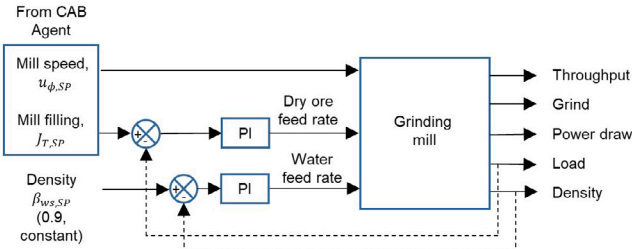


Fig. 3. Simulation plant of a grinding mill with two regulatory controllers.
Source: Reproduced from Ziolkowski et al. (2022)

2.2.1. Process description

Table 1 provides descriptions of each of the variables indicated in Fig. 3. The table indicates whether the variable is a manipulated variable (MV) or controlled variable (CV) for the optimisation experiments conducted in this work. A distinction is made between manipulated variables process measurements, e.g. mill speed, and their setpoints (SP), e.g. mill speed setpoint, which is what the optimiser would ultimately manipulate.

The simulation plant in Fig. 3 represents a typical grinding-mill operation with regulatory control. The grinding mill is fed continuously with dry ore and water. Most mills contain additional grinding media to further aid in particle breakage. The tumbling action of the charge causes breakage primarily through impact and attrition between ore particles and grinding media. Grinding mills are typically operated to achieve maximum throughput while maintaining discharged particles

below a specification size (Wills and Napier-Munn, 2006, p. 176). However, these goals conflict since higher throughputs require shorter residence times inside the mill, resulting in less size reduction while travelling through the mill.

Practitioners generally control three MVs: the dry-ore feed rate, water feed rate, and mill speed. As the figure demonstrates, the dry-ore feed rate is usually coupled with the mill filling to control the filling to a setpoint. The water feed rate is manipulated to control the discharge density. Regulatory control on the filling allows for further use of this variable as an MV in advanced throughput and grind optimisation. If the mill is equipped with a variable-speed drive, the mill speed becomes another parameter available for throughput and grind optimisation.

The dynamic response of the simulation plant to step changes in the mill filling and speed are shown in Fig. 4. -10 dB of white noise was added to the throughput, and -50 dB noise was added to both the filling and grind measurements. Both ideal and noisy responses are shown in Fig. 4 in blue and red, respectively. The simulation is based on a sixty-second sampling time. Fig. 4 demonstrates that the dynamics of the performance indicators are slow, and the process reaches a steady state approximately two hours after steps in the filling or speed.

2.2.2. Steady-state grind curves

Le Roux et al. (2020) calibrated the grinding-mill model with industrial grind-curve data previously reported by Van der Westhuizen and Powell (2006). The resultant model could replicate short-term grinding-mill dynamics with steady-state behaviour governed by grind curves. Ziolkowski et al. (2021, 2022) investigated controller testing using the calibrated model. Le Roux et al. (2013, 2020) present detailed analyses of the model's steady-state behaviour and dynamic responses.

Table 1
Simulation plant variable descriptions and their MV/CV designation for optimisation experiments.

Variable	Symbol	Description	MV/CV	Unit
Dry ore feed rate	u_o	Dry ore feed rate	–	t/h
Water feed rate	u_w	Fresh water feed rate	–	m ³ /h
Mill filling	J_T	Volumetric fraction of mill filled with charge	–	–
Mill filling setpoint	$J_{T,sp}$	Mill filling setpoint as specified by an optimiser or operator	MV	–
Mill speed	u_ψ	Mill rotational speed expressed as a fraction of the speed required to centrifuge the charge.	–	–
Mill speed setpoint	$u_{\psi,sp}$	Mill speed setpoint as specified by an optimiser or operator	MV	–
Throughput	Q_s	Volumetric flow rate of solids discharged from the mill.	CV	m ³ /h
Grind	ϕ	Volumetric fraction of discharged solids below specification size	CV	–
Power	P_M	Mill motor power draw	CV	kW
Density	β_{ws}	Water to solids ratio of discharged slurry.	–	–
Density setpoint	$\beta_{ws,sp}$	Water to solids ratio of discharged slurry setpoint (constant)	–	–

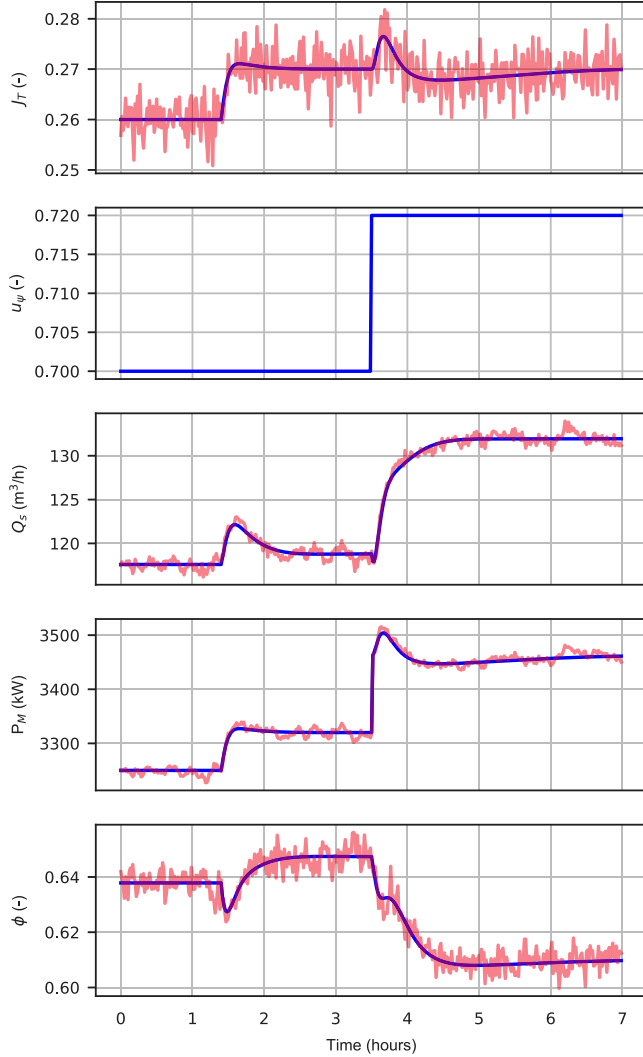


Fig. 4. Dynamic response of simulation plant in Fig. 3 to a step change in mill filling (time = 1 h) and mill speed (time = 3.5 h). The ideal (noiseless) response is shown in blue. Red signals are contaminated with measurement noise.

Fig. 5 presents the steady-state response surfaces for mill-performance indicators generated by the grind-curve model. The data was generated by sweeping the calibrated model across filling values in the open interval [0.20, 0.45] and speed values in the open interval [0.60, 0.75], with no additive noise. The CAB agent will traverse these surfaces in search of the optimums, indicated in yellow on each figure.

Fig. 5 clearly shows the regions of the action space wherein the optimal reward for a given performance indicator would be received.

Table 2

Settling times of mill-performance indicators to step changes in MVs used for optimisation.

	MVs					
	Filling			Speed		
	% to final steady-state value					
	99	90	63.2	99	90	63.2
CVs	Settling times (minutes)					
Throughput	80	65	50	95	54	17
Grind	82	59	38	216	52	37
Power	57	22	5	210	91	1

Moreover, the figure demonstrates that the optimal regions do not necessarily overlap. In the extreme case, the region producing the optimal grind generates the least throughput, demonstrating the counteracting nature of these variables.

2.3. Agent description

2.3.1. Agent step frequency

In the dynamic-simulation model, any setpoint changes are followed by transient periods due to controller action and settling times of the controlled process. Similar to human process operators, the agent needs to wait for the controllers to steer the process to the new setpoints and for the process to respond, to a degree, to the new setpoints before making another change. The agent step frequency N_{sf} was used as a hyper-parameter to control the ratio of steps by the environment per step made by the agent. Only the final measurement at time $t + N_{sf}$ is added to the agent's replay buffer prior to the agent making a step.

Performance indicator measurements taken during transient periods might not follow the steady-state maps in Fig. 5. These measurements could prevent the agent from recovering the unknown grind curve. Accordingly, one might expect performance improvements by minimising the amount of transient data in the agent's replay buffer. The closed-loop settling times of the performance indicators to step changes in the mill filling and speed could provide an initial estimate of the appropriate N_{sf} to achieve this. The settling times of performance indicators after step changes in the two MVs considered for optimisation are summarised in Table 2. The settling times were approximated at different percentages to the final steady-state values.

Determining the degree of settling to the steady state required for approximating the grind curve is not trivial and affects the convergence time. Longer N_{sf} would generate a training dataset conforming closer to the maps in Fig. 5, but the agent would take fewer actions per unit of time. This trade-off required an investigation of the sensitivity of the rewards achieved by the agent to the N_{sf} .

2.3.2. Time-series data storage

As demonstrated in Fig. 2, the agent trains new regression models at each step from past experiences. Consequently, the agent requires a

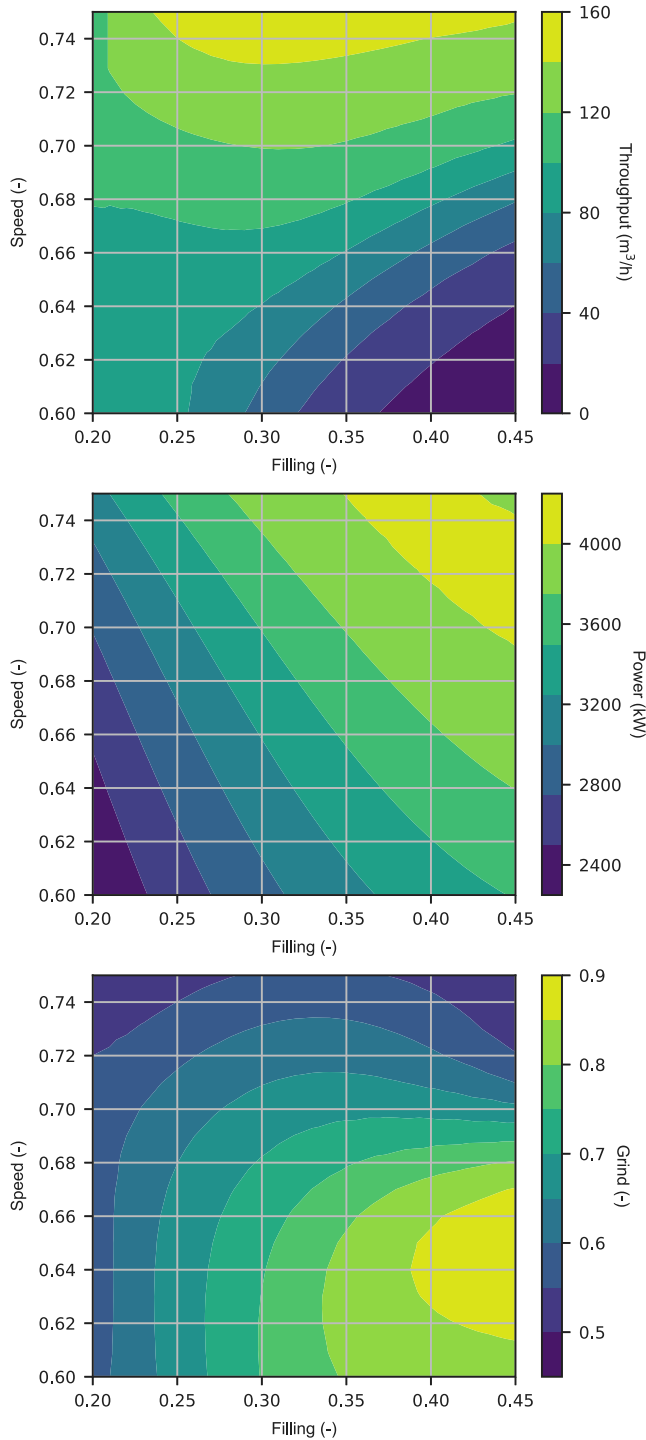


Fig. 5. Contour plots depicting the steady-state response surfaces for the (top) throughput, (middle) power and (bottom) grind for the simulation plant in Fig. 3.

replay buffer to store such experiences and samples from these experiences during training. In this work, the length of the replay buffer was limited to 100 samples. The buffer is a First-In-First-Out (FIFO) queue, with the oldest samples being removed once the hundred-sample limit is reached. All regression and classification models generate a mapping between the action space and different mill-performance indicators or constraints. Consequently, the models are trained on different datasets requiring multiple replay buffers. In total, $N_R + 2N_b$ replay buffers are maintained at all times.

At the start of each step, the replay buffer of each regression model receives a vector $\langle \omega(\mathbf{a}_t), R_{j,t+N_{sf}} \rangle$, $j \in \mathbb{Z}^{1 \leq j \leq N_R}$, each containing a different real-value reward component. ω indicates an expansion of the feature vector, as detailed in the following section. In contrast, each classification model has two replay buffers, separating positively- and negatively-classified samples. These buffers receive data conditionally based on the value of the binary reward component in $\langle \mathbf{a}_t, R_{j,t+N_{sf}} \rangle$, $j \in \mathbb{Z}^{N_R \leq j \leq N_R+N_b}$, generated at each step. With constraint violations expected to be rare, the negatively-classified buffer indicating non-violations should overflow regularly. In contrast, the positively-classified buffer will populate slower and help the model remember constraint violations for longer periods. Models are trained on all samples currently available in the replay buffer.

The grinding-mill process is not stationary. Over shorter timescales, feed ore disturbances frequently perturb the system to new operating points. Over longer timescales, changes in the grinding-media loading and liner degradation slowly shift the operation. The fixed-size replay buffer was selected to ensure the agent only uses the most recent data. This ensures that data used for model training represents the system's state when inference is required.

2.3.3. Robust regression of real-value reward components

Multiple linear regression was used to model the polynomial relationship between mill performance indicators in the reward vector $\mathbf{R}_{t+N_{sf}}$ and the action space. Linear regression fits a model of the form in Eq. (3) between the feature vector $\mathbf{x}$ and a response variable $f_j(\mathbf{x})$ or y_j ; with β_j^T representing the weights vector and β_0 a constant offset:

$$f_j(\mathbf{x}) = \beta_j^T(\mathbf{x}) + \beta_0; \quad j \in \mathbb{Z}^{1 \leq j \leq N_R} \quad (3)$$

In this work, the set of possible actions the agent can manipulate becomes the feature vector, and the response variable is a real-value reward component, $R_{j,t+N_{sf}}$, obtained from the simulation environment. The feature set was expanded from the original action space with polynomial basis-functions to account for the known quadratic relationships and interaction between the mill filling and speed. The expanded feature set is provided in Eq. (4):

$$\begin{aligned} \mathbf{x} = \omega(\mathbf{a}_t) &= \omega(\{J_{T,sp}, u_{\phi,sp}\}) \\ &= \{J_{T,sp}^2, u_{\phi,sp}^2, J_{T,sp}u_{\phi,sp}, J_{T,sp}, u_{\phi,sp}, 1\} \end{aligned} \quad (4)$$

Model weights are grouped according to $\theta_j = [\beta_j, \beta_0]^T$. The functional form of the linear regression implemented by the agent is given in Eq. (5). Consequently, data from the environment enters the agent's replay buffer as a vector of $\langle \omega(\mathbf{a}_t), R_{j,t+N_{sf}} \rangle$ samples. A regression model is trained for each of the N_R real-value reward components in $\mathbf{R}_{j,t+N_{sf}}$:

$$R_{j,t+N_{sf}} = \theta_j^T \omega(\mathbf{a}_t); \quad j \in \mathbb{Z}^{1 \leq j \leq N_R} \quad (5)$$

This work used the iteratively reweighted least-squares algorithm with Huber's t-function (Huber, 1964) as a robust criterion estimator, as implemented in the statsmodels Python library (Seabold and Perktold, 2010). This approach assigned lower weights to samples classified as outliers. Shipman (2021) notes that ordinary and weighted-least squares were unsuitable for the amount of noise contamination while operating in a similar environment. Shipman (2021) deemed the robust regression necessary to enhance model accuracy with the noise levels generally found in process signals.

In an operational milling circuit, θ_j^T is a function of time t owing to variability in several properties, including the ore properties and liner behaviour owing to wear. Using rolling regression, as done here with a window of 100 data points, ensures that the agent is not biased by a model of the historical behaviour of the system that is no longer valid. An alternative approach is to perform time-varying parameter Bayesian linear regression using Markov Chain Monte Carlo (MCMC) methods (Bitto and Frühwirth-Schnatter, 2019; Cadonna et al., 2020; Knaus et al., 2021; Berninger et al., 2022). However, these methods are significantly slower than the iteratively reweighted least-squares algorithm used here. Therefore, they were unsuitable for this work's large volume of tests.

2.3.4. Logistic regression of binary reward components

The environment detects constraint violations and communicates the outcome through binary components in the reward vector. A violated constraint will return a reward of 1, while non-violations return 0. To avoid violating such a constraint again, the agent trains logistic regression models mapping the action space to the binary reward. The decision function of the model is used as an inequality constraint during the optimisation phase to avoid repeating actions leading to constraint violations.

Logistic regression models are linear classifiers that learn models of the form given in Eq. (6). The logistic-regression models map from the original action space rather than the polynomial expansion in Eq. (4). The output variable is squashed into a range [0, 1] using the sigmoid function in Eq. (7). A threshold in the [0, 1] range is applied to determine to which class a sample belongs, as given in Eq. (8):

$$g_j(\mathbf{a}_t) = \mathbf{w}_j^T \mathbf{a}_t + b_j \quad (6)$$

$$\hat{p}_j(\mathbf{a}_t) = \frac{1}{1 + e^{-g_j(\mathbf{a}_t)}} \quad (7)$$

$$y_j = \begin{cases} 1, & \hat{p}_j(\mathbf{a}_t) \geq 0.5 \\ 0, & \hat{p}_j(\mathbf{a}_t) < 0.5 \end{cases} \quad (8)$$

In the two-dimensional action space used here, the model defines a straight line that, once crossed, indicates a constraint was violated. Without the sigmoidal activation and thresholding, Eq. (6) evaluates to a value proportional to the distance of a sample to the line or hyperplane defined by the model. This scalar distance is used during the optimisation step to determine how far a candidate action is from violating a constraint.

The constraint models were implemented using the *LogisticRegression* class from *scikit-learn*'s (Pedregosa et al., 2011) *linear_model* module in Python. *Scikit-learn*'s implementation minimises the cost function in Eq. (9) with regularisation using the l^2 -norm:

$$\min_{\mathbf{w}_b} C \sum_{i=1}^n (-y_i \log(\hat{p}_j(\mathbf{a}_i)) - (1 - y_i) \log(1 - \hat{p}_j(\mathbf{a}_i))) + \frac{1}{2} \|\mathbf{w}_j\|_2^2 \quad (9)$$

With constraint violations expected to occur less frequently than non-violations, the model-training data set is expected to be imbalanced. Larger penalties were imposed for misclassifying constraint violations (false negatives) to address the imbalanced data. The penalties were weighted inversely proportional to the class frequencies of the training data, where samples from classes with lower frequencies produce a higher cost when misclassified. Additionally, the C -parameter in Eq. (9) was set to 1000 to effectively ignore the regularisation terms and produce classifiers with high recall.

The agent maintains two replay buffers with a maximum of 100 samples each for classifier training. One buffer is for samples that do not violate a constraint, while the other retains constraint-violation samples. Since constraint violations are expected to be rare, this procedure helps retain the constraint violation and model for extended periods. If the agent has not yet observed a violation, the model cannot be trained, and the constraint is ignored.

2.3.5. Thompson sampling

This work used Thompson Sampling (Thompson, 1933) to explore the action space for the real-value reward models. Thompson sampling is an exploration method based on Bayesian inference used in multi-armed, contextual and CAB problems (Agrawal and Goyal, 2012a). Latimore and Szepesvári (2020) and Burtini et al. (2015) describe Thompson sampling for CABs with linear pay-offs. The main steps of the algorithm are described below.

Thompson sampling starts by assuming a prior distribution over the parameter vector θ_j of each regression model generated in Eq. (5). A Gaussian prior was assumed and generated using the parameter vector

θ_j and its covariance calculated using Huber's estimator (Huber, 1981, p. 173). Eq. (10) gives the prior distribution:

$$p(\theta_j) = \mathcal{N}(\theta_j, \text{cov}(\theta_j)) \quad (10)$$

Usually, Thompson sampling updates the prior distribution with the set of experiences $\mathcal{D}$, consisting of samples of $\langle \omega(\mathbf{a}_t), \mathbf{R}_{t+N_{sf}} \rangle$, to yield a posterior distribution using Eq. (11). As noted in Section 2.3.3, this work uses an iteratively reweighted least-squares algorithm with a rolling window of data in place of the Bayesian linear-regression update process in Eq. (11). However, the core principle of Thompson sampling, i.e. sampling a random set of model parameters, given the mean and covariance estimates of those parameters, remains in use. Section 2.3.3 explains the motivation for this choice.

$$p(\theta_j | \mathcal{D}) \propto p(\mathcal{D} | \theta_j) p(\theta_j) \quad (11)$$

A candidate parameter vector is sampled from the posterior distribution in Eq. (12):

$$\hat{\theta}_j \sim p(\theta_j | \mathcal{D}) \quad (12)$$

Intuitively, Thompson sampling uses the uncertainty in the linear model parameters to explore the action space. High covariance in the linear model results in a high variance in $p(\theta_j)$, which causes more considerable changes in the sampled model parameters $\hat{\theta}_j$ between successive agent steps. The action $\mathbf{a}_t$ for the next agent step is determined by solving the optimisation problem defined in Eq. (13):

$$\arg \max_{\mathbf{a}_t} \hat{\theta}_j^T \omega(\mathbf{a}_t) \quad (13)$$

Eq. (13) applies if there is a single reward component, e.g. throughput, and no constraints. This corresponds to the classical CAB problem. Section 2.3.6 expands upon this to show how constraints and a non-linear combination of the reward components are handled.

2.3.6. Action selection

The real-value reward-component models, now parameterised by the parameter vectors $\hat{\theta}_j \in [\hat{\theta}_j]$ sampled in Eq. (12), are aggregated into a scalar-reward function according to Eq. (14):

$$\mathcal{R}_{t+N_{sf}} = \lambda(\mathcal{L}([\hat{\theta}_j], \omega(\mathbf{a}_t))) \quad (14)$$

The objective function $\lambda(\mathcal{L})$ can be a linear or non-linear function of the plant CVs and can be used to achieve key performance indicators such as maximising profit or limiting water and electricity usage. Furthermore, time-varying components such as dynamic electricity tariffs or water rates that are known beforehand can be added to $\lambda(\mathcal{L})$.

The agent selects the following action by solving the optimisation problem defined in Eq. (15). The Sequential Least-Squares Programming (SLSQP) algorithm (Kraft, 1988), as implemented in Python's SciPy package (Virtanen et al., 2020), was used to solve the optimisation problem:

$$\max_{\mathbf{a}_t} \mathcal{R}_{t+N_{sf}} \quad (15a)$$

$$\text{subject to } \mathbf{w}_j^T \mathbf{a}_t + b_j \geq 0; j \in \mathbb{Z}^{1 \leq j \leq N_b} \quad (15b)$$

$$\mathbf{a}_l \leq \mathbf{a}_t \leq \mathbf{a}_u \quad (15c)$$

$$|\mathbf{a}_t - \mathbf{a}_{t-1}| \leq \delta(\mathbf{a}_u - \mathbf{a}_l) \quad (15d)$$

Eq. (15b) represents learned constraints from the logistic-regression classifiers. Eq. (15c) represents user-supplied bounds on the actions. The upper $\mathbf{a}_u$ and lower $\mathbf{a}_l$ limits on the action space would typically constrain the agent to operate inside equipment and safety limits. Eq. (15d) limits the magnitude of setpoint changes between successive agent steps, expressed as a fraction of the difference between the upper and lower action limits defined in Eq. (15c). The parameter δ , referred to here as the agent step size, is considered another hyper-parameter of the agent. The step size affects the rate of convergence of the

agent to the optimal operating point. However, the step size must also remain low enough to avoid large variable movements from causing disturbances to downstream processes.

The complete algorithm is now formalised in Algorithm 1:

Algorithm 1 CAB Algorithm in Grinding-Mill Simulation Environment

Input: $N_{\mathbb{R}}, N_b, N_{sf}, \delta, \mathcal{P}_e$

- 1: Initialise simulator with a pseudo-random number generator seed $\mathcal{P}_e$.
- 2: Set $\mathbf{a}_t$ to current process setpoints.
- 3: **for** $t = 1, 1 + N_{sf}, \dots$, end of simulation **do**
- 4: Setpoints in $\mathbf{a}_t$ are sent to the environment's simulator in Fig. 3.
- 5: **for** $i = 1, 2, \dots, N_{sf}$ **do**
- 6: Loop the simulator with new setpoints.
- 7: **end for**
- 8: Environment builds the reward vector $\mathbf{R}_{t+N_{sf}}$ according to Eq. (2).
- 9: Add $\langle \omega(\mathbf{a}_t), \mathbf{R}_{t+N_{sf}} \rangle$ to the set of experiences $\mathcal{D}$ in the agent time-series data store.
- 10: **for** $j = 1, 2, \dots, N_{\mathbb{R}}$ **do**
- 11: Model $R_{j,t+N_{sf}}$ using Eq. (5) and experience $\mathcal{D}$.
- 12: Generate a Gaussian distribution $p(\theta_j|\mathcal{D})$ over θ_j , according to Eq. (10)
- 13: Sample a candidate parameter vector $\hat{\theta}_j$ from $p(\theta_j|\mathcal{D})$, Eq. (12).
- 14: **end for**
- 15: Aggregate all $\hat{\theta}_j$ according to Eq. (14) to get scalar function $\mathcal{R}_{t+N_{sf}}$.
- 16: **for** $j = N_{\mathbb{R}} + 1, \dots, N_{\mathbb{R}} + N_b$ **do**
- 17: **if** $R_{j,t+N_{sf}}$ indicates a constraint was violated **then**
- 18: Fit a decision surface defining the constraint using Eq. (6) and $\mathcal{D}$.
- 19: **end if**
- 20: **end for**
- 21: Solve the optimisation problem in equation (15) using SLSQP to calculate $\mathbf{a}_t$ for the next agent step.
- 22: **end for**

2.4. Agent hyper-parameter selection

The CAB agent has two hyper-parameters namely N_{sf} and δ . While one could obtain some intuition on the optimal N_{sf} by analysing process time constants or the settling times in Table 2, the effects of the interaction between N_{sf} and δ were unknown. Larger δ is expected to result in longer transient periods, requiring longer N_{sf} for optimisation. However, longer N_{sf} mean fewer actions per unit of time and will increase the time to convergence at the optimum. A search of the agent hyper-parameter space was warranted.

A grid search was done for each experiment to produce maps of the reward achieved as a function of the hyper-parameters. In addition to identifying the optimal parameter set, a grid search allows analysis of the algorithm's sensitivity to the hyper-parameters.

Algorithm 2 defines the hyper-parameter grid search routine. Each hyper-parameter set was trialled in 100 episodes, each initialised with different pseudo-random number-generator (PRNG) seeds. These seeds were used to initialise the random number-generators for the environment noise and for sampling from the Gaussian distribution of candidate models, indicated in Eq. (12). The average reward achieved across the 100 episodes was used to quantify the performance of each parameter set. For all optimisation experiments, the search space was confined to the following regions:

- δ : the closed interval of [0.01, 0.10] in increments of 0.005. This results in maximum manipulated variable movements in the range of 1% to 10% of the designated action limits.

- N_{sf} : the closed interval of [10, 80] in increments of two. This corresponds to periods of 10 to 80 min between successive agent steps.

The grid search ranges were determined in an initial search using a Bayesian minimisation routine. This method is more appropriate for the larger search space, especially considering the computational cost of evaluating each hyper-parameter pair. This search explored a larger range of N_{sf} of [1, 120], thereby covering most settling times in Table 2. Maximum results were consistently observed at step frequencies below 80 min, and the grid search space was reduced.

Algorithm 2 CAB Algorithms Hyper-Parameter Grid Search

Input: $N_{\mathbb{R}}, N_b, \mathcal{A}_l, \mathcal{A}_u, \mathcal{A}_\Delta, S_l, S_u, S_\Delta, \mathcal{P}$

- 1: Initialise vector $\mathcal{A}$ of evenly-spaced N_{sf} values between $\mathcal{A}_l$ and $\mathcal{A}_u$ with step size $\mathcal{A}_\Delta$.
- 2: Initialise vector $\mathcal{S}$ of evenly-spaced δ values between S_l and S_u with step size S_Δ .
- 3: Initialise pseudo-random number generator with seed $\mathcal{P}$.
- 4: **for all** $N_{sf} \in \mathcal{A}$ **do**
- 5: **for all** $\delta \in \mathcal{S}$ **do**
- 6: **for** $i = 1, 2, \dots, 100$ **do**
- 7: Set episode seed: $\mathcal{P}_e = \mathcal{P} + 1000 \times i$
- 8: Run Algorithm 1 with $(N_{\mathbb{R}}, N_b, N_{sf}, \delta, \mathcal{P}_e)$.
- 9: Calculate and record average reward for the episode.
- 10: **end for**
- 11: Calculate and record the average reward across all 100 episodes.
- 12: **end for**
- 13: **end for**

3. Results

This section presents the results of three optimisation experiments: mill-throughput optimisation, a weighted throughput-grind objective function and a comprehensive-profit function. The first two experiments were configured similarly to those in Ziolkowski et al. (2022). The final experiment is an extension of the throughput optimisation experiment with operational costs being considered. Eq. (16) defines the action limits for all experiments based on the available grind-curve data:

$$\begin{aligned} \mathbf{a}_l &= \langle J_{T,l}, u_{\phi,l} \rangle = \langle 0.20, 0.60 \rangle \\ \mathbf{a}_u &= \langle J_{T,u}, u_{\phi,u} \rangle = \langle 0.45, 0.75 \rangle \end{aligned} \quad (16)$$

3.1. Throughput optimisation

This subsection describes the throughput-optimisation experiment, detailed in Section 6.2.2 in Ziolkowski et al. (2022). For this experiment, the environment builds the following reward vector during each agent–environment step:

$$\mathbf{R}_{t+N_{sf}} = \langle \text{throughput}_{t+N_{sf}}, \text{binary}_{\text{grind} < 0.5, t+N_{sf}}, \text{binary}_{\text{grind} > 0.93, t+N_{sf}} \rangle \quad (17)$$

For throughput optimisation, the agent requires the throughput in the reward to train a model, mapping the throughput achieved to the action space. The two binary-reward components will take on a value of one, if either is violated, prompting the training of a constraint model. With only a single real-value reward component required, no aggregation of real-value reward components according to the objective function in Eq. (14) was needed. The scalar reward function $\mathcal{R}_{t+N_{sf}}$ consists solely of the throughput grind curve estimate.

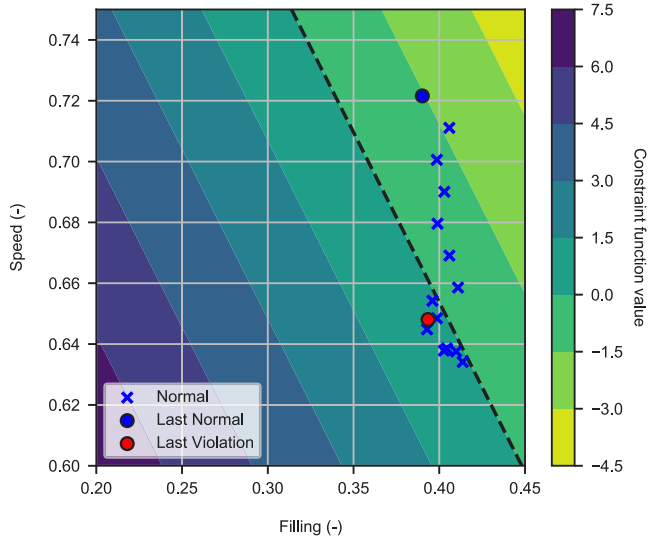


Fig. 6. Linear constraint in the 2D action space, trained during a throughput optimisation experiment.

3.1.1. Constraint specification

The experiments in Ziolkowski et al. (2022) did not require constraints. This work added upper and lower limits on the grind as constraints. Once these limits are exceeded, a constraint model is trained and implemented in the optimisation.

The grind was constrained to a range of [0.50, 0.93], the extremums defined by the grind curve data. The authors noted during testing that these limits could be violated, warranting the addition of constraints. Furthermore, the dynamic model is unstable at particular combinations of high filling and high speed values, which could cause the grind to violate physical limits (Le Roux et al., 2020). These constraints would also prohibit the agent from veering to this region in the action space.

An example of a trained constraint model representing the upper grind limit during the throughput-optimisation experiment is provided in Fig. 6. The agent trajectory starts on the bottom right of the figure at high filling and low speed. Extended operation in this region eventually violates the upper grind limit, indicated by the red dot. A trained logistic-regression model is used as a constraint in the optimisation algorithm during the next agent step. The constraint forces the following action to change the direction of the agent's trajectory through the action space towards the constraint boundary. Accordingly, the agent starts increasing the mill speed until crossing the constraint boundary. The agent's search continues, eventually settling at a lower filling and higher speed to maximise throughput.

3.1.2. Grid-search results and optimal-agent trajectory

The results of applying Algorithm 2 for the throughput optimisation experiment are given in Fig. 7. The figure shows the average reward (throughput) received over the 100 trajectories for each agent configuration. Large regions in the map produce quantitatively similar results. The highest average throughput of 144 m³/h was achieved with a (δ , N_{sf}) parameter set of (0.07, 48). This step size allows for 1.75% and 1.05% deviations in the mill filling and speed, respectively, when these variables are expressed as percentages instead of fractions.

Fig. 7 confirms the original expectations that larger step sizes require longer step frequencies to achieve comparable rewards. The worst-performing agents had large δ and short N_{sf} , or vice versa. Larger δ produces longer transient periods. Coupled with a short N_{sf} , the agent dataset diverges from the maps in Fig. 5, rendering the agent unable to recover the optimal curve. Conversely, small δ with long N_{sf} lead to a slower approach to the optimal point and lower throughput. The white region in the top left corner of Fig. 7 is characterised by large

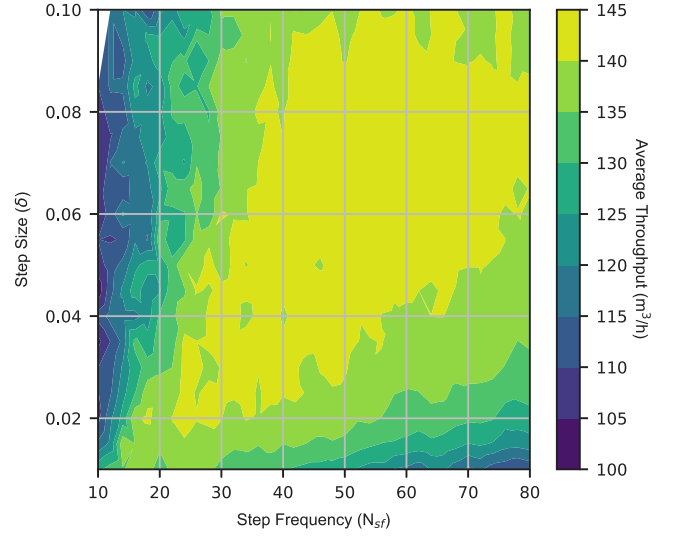


Fig. 7. Contour plot depicting hyper-parameter grid search results during throughput optimisation.

δ and short N_{sf} , leading to many episodes wherein the optimisation in Eq. (15) would regularly fail and was excluded from the results. Good average results are obtained at step frequencies of 30 min and upwards, significantly lower than the steady-state settling times in Table 2.

Fig. 8 displays the time series generated using the best-performing parameter set that achieved the highest reward alongside the results of Ziolkowski et al. (2022). Three different ESCs were tested by Ziolkowski et al. (2022), viz. a perturbation-based ESC (PESC), a time-varying parameter estimation ESC (TESC), and a Nelder–Mead simplex ESC (SESC). Further, Fig. 9 shows the best-performing agent's average trajectory through the action space, superimposed on the throughput grind curve.

Algorithm 2 results in 100 trajectories through the simulation for each agent configuration. Fig. 8 portrays the average behaviour of an agent and the variance between individual trajectories exhibited over the 100 runs. At each time step, the throughput across all 100 tests was collected, and the 5th percentile (P_5), average, and 95th percentile (P_{95}) values of the throughput were calculated. Visualising these statistics over time generates a hypothetical trajectory, representing the convergence rate to the optimal policy across the 100 episodes for each hyper-parameter configuration.

Figs. 8 and 9 demonstrate how the agent learns to maximise the mill throughput by increasing the speed to 0.75 and decreasing the filling to around 0.36. This is near the (speed, filling) configuration of (0.75, 0.35), where the theoretical maximum throughput of the model is achieved (Ziolkowski et al., 2022, Table 2), thereby proving that the agent is capable of finding regions near the unknown optimums. The CAB algorithm produces qualitatively similar variable responses to the ESCs tested by Ziolkowski et al. (2022) within similar timeframes.

Table 3 gives the average throughput achieved by each optimiser for different periods in the simulation. The table provides statistics for all three hypothetical trajectories. Exploration towards the optimal operating point occurs within the first 20 h, where the throughput is steady afterwards. The average CAB agent achieved a steady-state throughput similar to the ESCs. P_{95} statistics indicate some agents consistently achieve higher throughputs. Measurement noise prohibited the optimisers from finding the optimal solution, which would result in a steady-state throughput of 157 tonnes/h. Around 90 h into the simulation the CAB starts exploring again, leading to a slight drop in throughput. Most of the data from the first 20 h has been lost from the agent's replay buffer. Fewer unique points on the grind curve remain in the replay buffer, leading to a less reliable linear model and

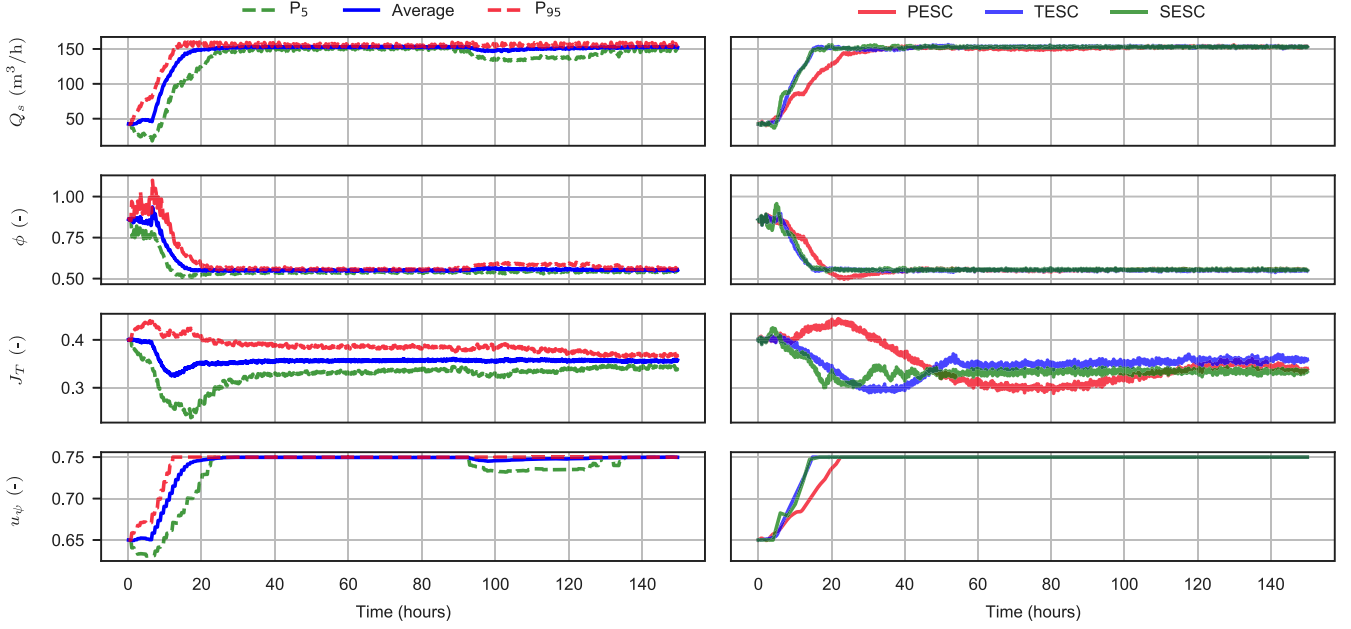


Fig. 8. Time series generated during throughput optimisation in (left column) this work and (right column) Ziolkowski et al. (2022). Legends apply to each column.

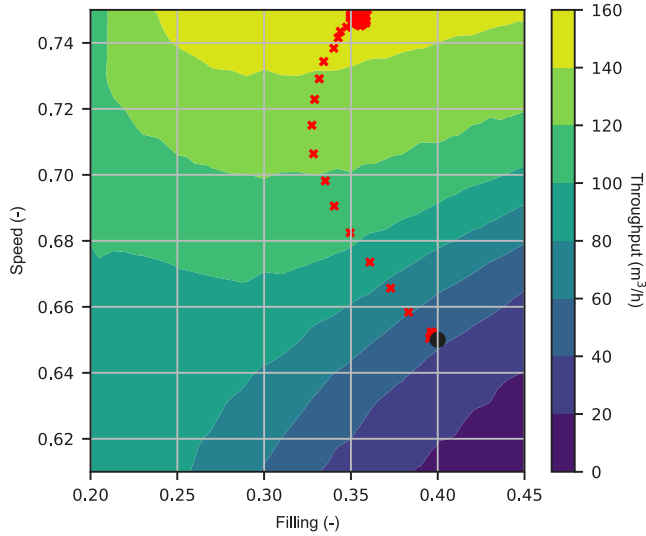


Fig. 9. Average trajectory of the best-performing agent through the action space for throughput optimisation. The agent's initial position is indicated with a black marker.

higher covariance in the parameter vector, ultimately causing further exploration.

In addition to the objective function values, Ziolkowski et al. (2022) quantified the performance of each optimiser using the convergence time. The convergence time was defined as the time the optimiser took to settle within 2% of the final steady-state values. However, under this definition, the CAB optimiser never converged throughout the experiment. Increased exploration compared to the ESCs meant the CAB agent consistently fluctuates outside the 2% threshold. Still, Fig. 8 qualitatively suggests that the optimisers converge in similar timeframes.

Fig. 10 shows the distribution of the average throughputs achieved in each of the 100 experimental runs at the optimal hyper-parameter configuration. This is an alternate method to analysing convergence times, since agents taking longer to converge to the optimum, would naturally report lower average throughputs. The average throughput

Table 3

Average throughput achieved in throughput-optimisation experiments.

		Hours			
		0–20	20–80	80–150	Total
CAB	P ₅	65.9	148.9	143.0	135.1
	Average	95.5	152.5	151.2	144.3
	P ₉₅	115.0	155.4	155.4	150.0
ESC	PESC	80.4	149.3	152.5	141.6
	TESC	99.8	152.1	153.2	145.6
	SESC	100.4	152.5	152.9	145.7

for each of the ESCs is also marked. The figure confirms that the majority of the agent instances outperformed PESC. Further, some instances also outperformed TESC and SESC. Others, while not outperforming TESC and SESC, still achieved satisfactory performances with the peak in the distribution near the performance of TESC and SESC.

3.1.3. CAB agent trajectory variance

Fig. 8 indicates that the CAB agent can show considerable variance in its behaviour between tests conducted with the same agent configuration but different PRNG seeds. Consider the mill filling in Fig. 1. The optimal filling is lower than the initial value. Some agents start by increasing the filling and speed, while others decrease the filling. While the agents initially follow different policies, the agents collectively approach the optimal policy asymptotically as the simulation progresses. While the agent produces variable responses in the manipulated variables, the same level of variance is not observed in the objective function, the throughput.

Fig. 11 presents the variance in throughput for all tested hyper-parameter configurations. The variance for a specific configuration was defined as the area between the 5th and 95th percentile curves in Fig. 8. Different throughputs are recorded due to agents following different policies. Consequently, lower throughput variance indicates agent configurations where episodes led to faster convergence to the policy the agent considered optimal.

Analysing Figs. 7 and 11, agents achieving higher rewards exhibit less variance. The result confirms that the agents achieving the highest rewards had the fastest convergence rates to the optimal or near-optimal policy.

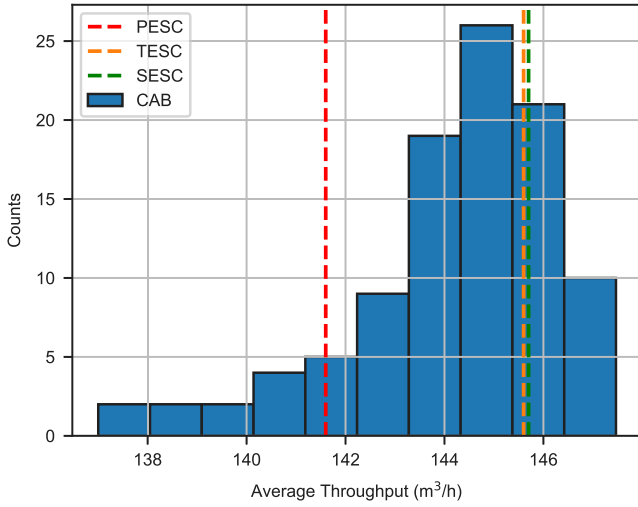


Fig. 10. Histogram of the average objective function value (throughput) for the 100 agents at the optimal hyper-parameter configuration. ESC performance is indicated by dashed lines.

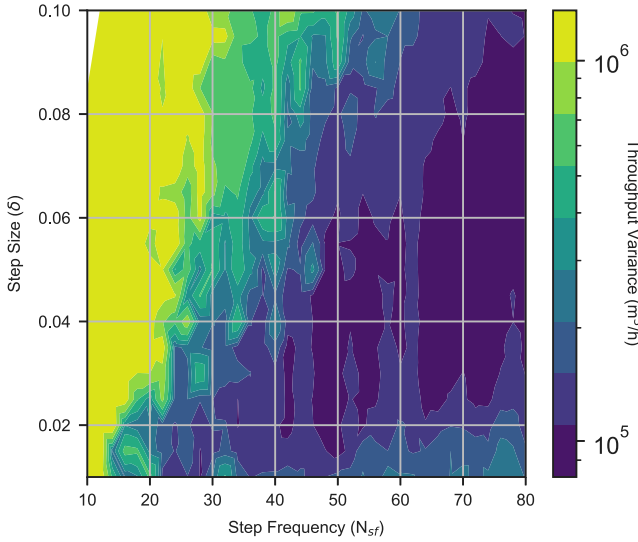


Fig. 11. Throughput variance produced by the CAB agent during throughput optimisation.

Table 4
Summary statistics of agent step times in seconds.

Minimum	Average	Maximum	Standard Deviation
0.034	0.649	0.909	0.138

3.1.4. Computational times

Summary statistics of the agent's step times for a single experimental run are presented in Table 4. These times include both the model training and optimisation steps. Table 4 demonstrates that average step times are well below a minute. A complete run of the simulation takes approximately 90 s. The algorithm can be run on modest computer hardware with almost no risk of calculations lasting longer than the agent's step frequency.

All experiments were run on a desktop computer with an AMD Ryzen Threadripper 3990X 64-Core Processor and 256 GB of RAM. Experiments are run in parallel in groups of 100 experiments. Therefore, each experiment is limited to a single core, with 72 experiments having to share a core with at least one other experiment. Figs. 7 and 11 were generated by testing 684 combinations of step size and

step frequency. Each combination was tested using a group of 100 experiments. Using the above-mentioned workstation, this process was completed in approximately seven hours as all 100 experiments in a single group are executed in parallel.

3.2. Weighted throughput-grind optimisation

This subsection describes the weighted objective-function experiments detailed in Section 6.3.1 in Ziolkowski et al. (2022). The objective function is defined by Eqs. (18)–(20):

$$J(Q_s, \psi) = \gamma J_{Q_s} + (1 - \gamma) J_\psi \quad (18)$$

$$J_{Q_s} = \frac{Q_s - Q_{s,min}}{Q_{s,max} - Q_{s,min}} \quad (19)$$

$$J_\psi = \frac{\psi - \psi_{min}}{\psi_{max} - \psi_{min}} \quad (20)$$

With Q_s and ψ referring to the throughput and grind, respectively, as indicated in Table 1; $Q_{s,min} = 0$, $Q_{s,max} = 158 \text{ m}^3/\text{h}$, $\psi_{min} = 0.5$ and $\psi_{max} = 0.93$, based on theoretical limits produced by grind curves. $J_{Q_s} \in [0; 1]$ and $J_\psi \in [0; 1]$ refer to the normalised throughput and normalised grind, respectively. γ denotes a weighting factor used to adjust the relative contributions of throughput and grind to the objective function. All results are shown for a γ value of 0.5. For this objective function, the environment builds the following reward vector during each agent–environment step:

$$\mathbf{R}_{t+N_{sf}} = \langle \text{throughput}_{t+N_{sf}}, \text{grind}_{t+N_{sf}}, \text{binary}_{\text{grind} < 0.5, t+N_{sf}}, \text{binary}_{\text{grind} > 0.93, t+N_{sf}} \rangle \quad (21)$$

Besides the throughput, this agent also requires the grind in the reward to train a model to map the grind to the action space. The objective function $\lambda(\mathcal{L})$, introduced in Eq. (14), is Eq. (18). The grind constraints introduced for the previous experiment are also used in this agent. Similar to Ziolkowski et al. (2022), the action space for this experiment consisted only of the mill filling. The reduced feature set is given in Eq. (22):

$$\omega(\mathbf{a}_t) = \omega(\{J_T\}) = \{J_T^2, J_T, 1\} \quad (22)$$

Fig. 12 displays the results of Algorithm 2 for the weighted throughput-grind optimisation. The white region in Fig. 12 is due to applying custom limits to the contour levels and some optimisation failures. The results of the grid-search are qualitatively similar for the two experiments. The highest average value of the objective function in Eq. (18) was achieved with a (δ, N_{sf}) parameter set of (0.04, 60). Again, large regions surrounding this configuration produce quantitatively similar results. These regions also had low variance, as in the throughput experiment. Fig. 13 displays the time series generated using this parameter set. Fig. 14 shows the best-performing agent's average trajectory through the action space, superimposed on the steady-state surface generated by the objective function in the action space. Results achieved by both optimisers are summarised in Table 5.

Fig. 14 shows that the objective function can achieve a maximum steady-state value of 0.57 from the initial setpoints when only the mill filling is available as a manipulated variable. Table 5 indicates that both optimisers reached a final steady-state value of 0.57, with similar final throughputs and grind achieved. The CAB agent convergence rates are significantly faster than the PSC optimiser. Fig. 13 provides qualitative confirmation of the speedier convergence time.

The faster convergence times are further portrayed in Fig. 15, with all CAB agents achieving a higher average objective-function value than the PESC optimiser.

Table 5 shows that the 5th percentile curve shown in Fig. 13 corresponds to a total objective function value of 0.53. Note that this curve shows the 5th percentile of the applicable variable at each point in time, i.e. five percent of the trajectories achieved a value below that curve at that particular point in time. However, none of the

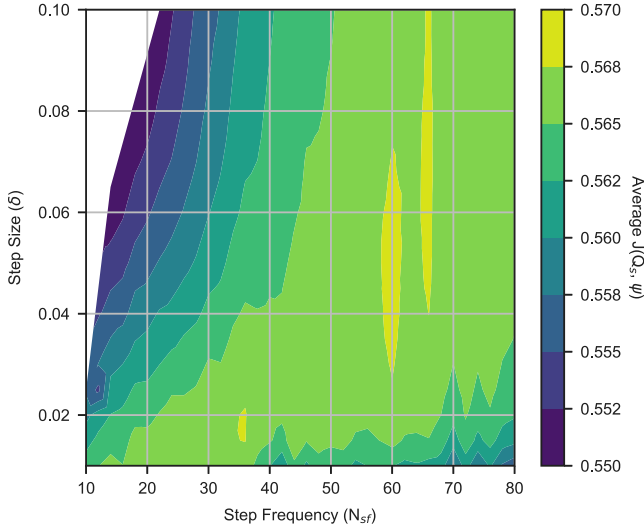


Fig. 12. Contour plot depicting hyper-parameter grid search results during weighted throughput-grind optimisation.

Table 5

Average objective function Eq. (18) values achieved in weighted throughput-grind optimisation experiments.

Optimiser	Hours					
	0–50	50–100	100–150	150–200	200–250	Total
CAB	Objective-function value - $J(Q_s, \psi)$					
	P_3	0.51	0.53	0.53	0.53	0.53
	Average	0.56	0.57	0.57	0.57	0.57
	P_3	0.60	0.61	0.61	0.61	0.61
ESC	PESC	0.51	0.56	0.57	0.57	0.57

trajectories in the hundred tests consistently remain outside of the 5%–95% confidence interval. Therefore, the total objective function value shown for the P_3 curve in Table 5 lies below the distribution of total objective function values for the tests, as shown in Fig. 15.

3.3. Maximising a profit function

This section expands on the experiments in Section 3.1 by considering the operational costs incurred when maximising mill throughput. This is to demonstrate that the CAB agent is capable of optimising functions with varying complexities.

Operational costs include an estimate of the cost of grinding media and liner wear, collectively called the grinding cost, and electricity costs subject to dynamic tariffs. Revenue from sales is calculated from the Net Smelter Return (NSR), as defined in Eq. (23) below. Furthermore, the Precious Metal Recovery (PMR) was modelled as a function of the grind achieved:

$$\text{NSR}_t = \text{throughput}_t \times \text{ore density} \times \text{ore grade} \times \text{PMR}_t \times \text{mineral price} \quad (23)$$

Undefined terms in Eq. (23) and constant values are summarised in Table 6. Note that the data in the tables and equations are not a representation of a specific milling circuit but rather an illustrative example to demonstrate the capabilities of the agent. The exact profit figures should not be compared to physical plants. The goal is to demonstrate that the agent can optimise a comprehensive objective function that accounts for revenue and costs.

During actual operation, the PMR is strongly dependent on the grind achieved. Like Wei and Craig (2009), the PMR was modelled as a quadratic function of the grind, as defined in Eq. (24). The polynomial

in Wei and Craig (2009) was adapted for increased effect on PMR over the ranges of grind values considered:

$$\text{PMR}_t(\psi) = -526.62 \times \psi_t^2 + 892.77 \times \psi_t - 305.29 \quad (24)$$

The grinding cost, C_G in USD/min, was modelled using the relation found in Thivierge et al. (2019) and repeated in Eq. (25) below:

$$C_{G,t} = (0.3 \times m_b P_{M,t} + 0.13 \times b_b P_{M,t}) \div 60 \quad (25)$$

$$m_b = 0.159(A - 0.015)^{0.34} \quad (26)$$

$$b_b = 0.0118(A - 0.015)^{0.3} \quad (27)$$

P_M refers to the mill power as reported in Table 1. In Eqs. (26) and (27), the variable A refers to the abrasion index of the ore, which was assumed to be constant at a value of 0.3. The degree of abrasiveness of the ore affects the wear rates of the grinding media and liners, which contribute to the operational costs of running a mill.

Electricity costs, $C_{E,t}$, were modelled as a piecewise function representing South African electricity tariffs, converted here to USD, as defined in Table 7. The remainder function uses the simulation index, t , measured per minute, to determine the specific time of day.

The agent is tasked with maximising the profit, as defined in the objective function in Eq. (28). The profit function requires the agent to estimate all three curves, namely throughput, grind and power. The grind constraints introduced in previous experiments were also retained.

$$\text{Profit}_t\left(\frac{\text{USD}}{\text{minute}}\right) = \text{NSR}_t - C_{G,t} - C_{E,t} \times P_{M,t} \div 60 \quad (28)$$

Instead of running another grid search for optimal agent hyper-parameter configurations, the optimal agent from Section 3.2 was used for this experiment. This was justified by the relative robustness of the agent's performance to hyper-parameters exhibited in previous experiments. The time series of noteworthy variables during the simulation are presented in Fig. 16.

Fig. 16 demonstrates the agent settling on average at a (filling, speed) combination of (0.34, 0.69). Both the filling and speed settled at lower values than in the throughput-optimisation experiment of Section 3.1. The agent's profit objective has shifted the focus more towards overall metal recovery, rather than simply maximising the throughput. This is achieved by operating at a lower speed, thereby increasing the residence time of the material to increase the grind closer to the point where maximum recovery is reached. This comes naturally with a decrease in the average throughput, yet more valuable mineral is finally extracted.

The dynamic electricity tariffs did not offset the possible gains from maximising the NSR term in Eq. (28). Consequently, while the agent considered these tariffs during optimisation, the optimal point did not move for the length of the simulation. The agent's trajectory on the profit-function map is depicted in Fig. 17.

Table 8 shows the average values achieved for the different components in Eq. (28) using this agent for profit maximisation. The table contains the same values for the throughput optimisation experiment if the system had been subjected to the same operational costs. This clearly demonstrates that the agent is able to adapt to a significantly more complex non-linear objective function in a similar amount of time as simpler objective functions, without requiring the retuning of the hyperparameters, demonstrating both robustness to hyper-parameter choice and broad applicability to a range of objective functions.

4. Discussion

4.1. CAB versus ESC

Both the ESCs and CAB agent successfully identified the unknown grind curves in the experiments in Section 3 and could steer the grinding mill to the vicinity of the optimum. The optimisers followed

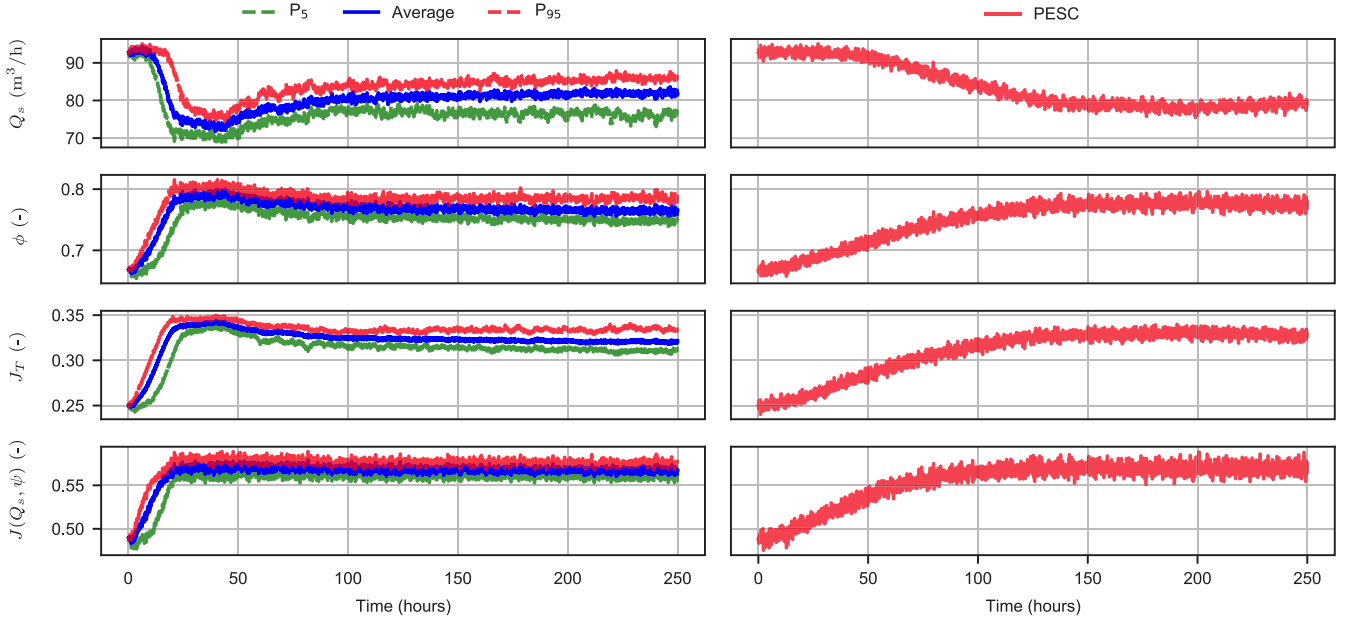


Fig. 13. Time series generated during weighted throughput-grind optimisation in (left column) this work and (right column) Ziolkowski et al. (2022). Legends apply to each column.

Table 6

Eq. (23) descriptions and constants.

Variable	Description	Value	Unit
NSR	Net Smelter Return	Calculated	USD/min
Ore density	Density of feed ores.	2.8	tonnes/m ³
Ore grade	Concentration of precious metal in feed ores.	2	grams/tonne
PMR	Fraction of precious metal in feed recovered by downstream separation processes.	Function of grind	–
Mineral price	Average gold price for the year 2023.	63	USD/gram

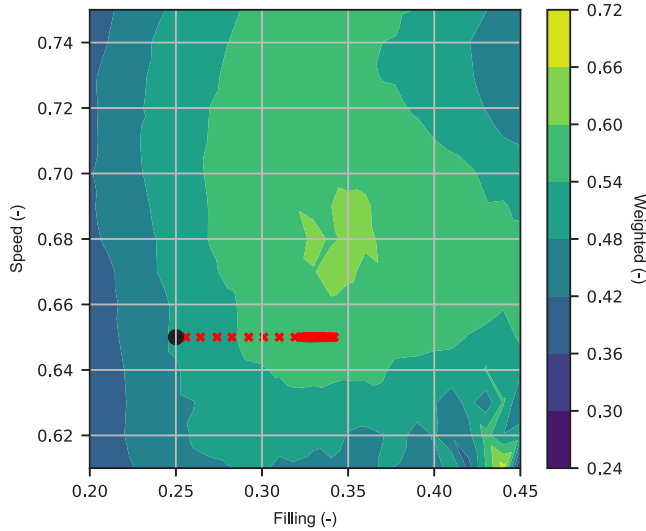


Fig. 14. Average trajectory of the best-performing agent through the action space for weighted throughput-grind optimisation. The agent's initial position is indicated with a black marker.

similar trajectories through the simulation, resulting in similar final steady-state values in comparable time frames.

The throughput-optimisation experiment demonstrated that the CAB agent could achieve a realistic optimisation objective, using the mill filling and speed. The average CAB agent achieved average throughputs

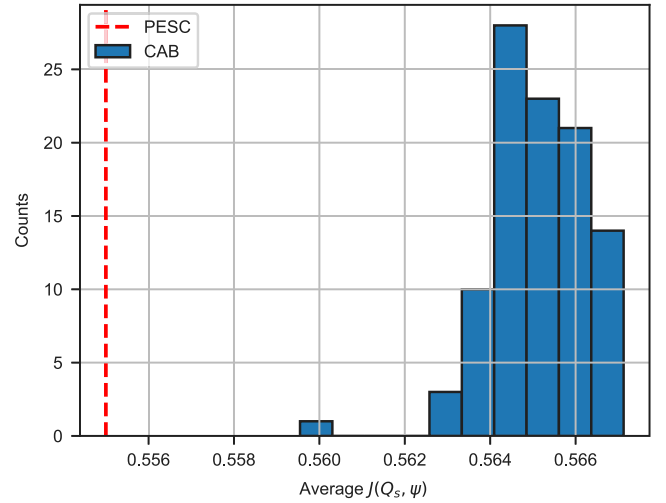


Fig. 15. Histogram of the average weighted throughput-grind objective function value for the 100 agents at the optimal hyper-parameter configuration. ESC performance is indicated using a dashed line.

1.9% higher than PESC, but 0.9% and 0.96% lower than TESC and SESC, respectively.

The weighted throughput-grind objective demonstrated how the objective function, Eq. (14), optimises arbitrary functions of multiple mill-performance indicators. Both optimisers settled at steady-state values that maximise the objective function when only filling is available

Table 7
Electricity costs $C_{E,t}$ as a function of the simulation index t .

$C_{E,t}$ (USD/kWh)	Applicable time periods
0.0355	$0 \leq t/60 \bmod 24 < 6$ $22 \leq t/60 \bmod 24 < 24$
0.0653	$9 \leq t/60 \bmod 24 < 17$ $19 \leq t/60 \bmod 24 < 22$
0.2156	$6 \leq t/60 \bmod 24 < 9$ $17 \leq t/60 \bmod 24 < 19$

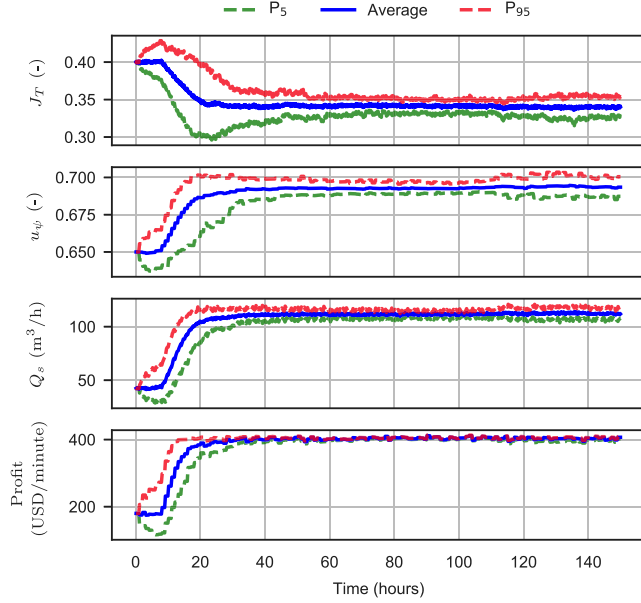


Fig. 16. Time series generate during optimisation of a profit function using the CAB agent.

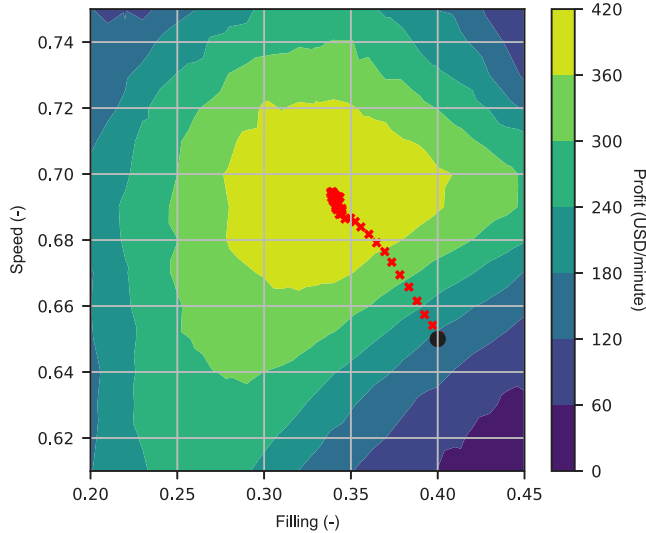


Fig. 17. Average trajectory of an agent with N_{sf} of 60 and δ of 0.04 through the action space for profit optimisation.

as MV. In this experiment, the average CAB agent yielded a 1.8% increase in the average performance function value. Fig. 13 qualitatively confirmed that the CAB agent yielded faster convergence times.

The probabilistic nature of the CAB trajectories hindered the direct comparison of metrics with the ESC. Some trajectories will achieve

higher rewards than the ESCs, while others will achieve less. However, the distribution of trajectories was necessary to demonstrate how the individual agents start with different policies that eventually converge to a single policy near or at the optimum.

4.2. Effect of the agent hyper-parameters

The CAB agent has two hyper-parameters that require tuning. However, as was demonstrated in the grid searches in Section 3, the two parameters are not necessarily independent. Larger step sizes allow the agent to make larger changes between successive steps. However, the step frequency also needs to be increased to allow the controllers to steer the process to the new setpoints and the process to respond.

The hyper-parameter grid searches revealed that agents could steer the process near the optimal operating point with step frequencies shorter than the closed-loop settling times. This is significant, considering one of the main limitations of the algorithm is the reliance on the process reaching a steady state between successive agent steps. This result demonstrates that the agent is capable of steering the process near the optimal operating point, even when the incoming data is not entirely steady.

In both experiments, one would obtain satisfactory results with step frequencies half of the 63.2% settling time, intended to represent the time constant of a first-order process. The optimal configuration in this work lies close to the 63.2% and 90% settling times of the filling vs throughput and the 90% settling time for speed vs throughput. Another general rule of thumb is to allow for three residence times for the mill to settle (Powell et al., 2009). At maximum throughput, this mill produces three residence times in about 80 min. From the simulation results, one could utilise the CAB agent within shorter intervals. However, by using the residence-time heuristic method, it would also produce quantitatively similar results.

Furthermore, the grid searches revealed large regions in the hyper-parameter space that produce agents with similar optimisation performance, thereby demonstrating a degree of robustness of the agent to these parameters. There were large regions of the hyper-parameter maps that would lead to near-optimal performance in both experiments, implying hyper-parameter sets could be used across experiments.

The CAB agent requires fewer hyper-parameters to be tuned than the ESCs, which require tuning between 4–7 parameters. Furthermore, the two hyper-parameters are highly intuitive and similar to the considerations of human process operators, when making decisions on control actions. The experiments conducted here indicate a simple heuristic method of selecting N_{sf} as the time constant of the slowest-responding performance indicator would suffice. However, this will need to be confirmed by testing on more systems.

Selecting the step size will require consideration of the amount of disturbance that can be tolerated downstream. This effect was not investigated in the experiments and requires consideration for industrial implementation. Ideally, a supervisory control system such as an MPC is available to stabilise the system at the desired setpoints.

4.3. CAB exploration

The throughput-optimisation experiment demonstrated how the CAB agent started exploring near the optimum after a prolonged period of steady operation. The event was triggered by a replay buffer full of steady-state data at similar operating points, and the model could not reliably fit the grind curve, triggering further exploration. This exploration is inherent to Thompson sampling and remains a limitation of the CAB agent.

Thompson sampling continuously forces the agent to explore the action space to different extents. The degree of exploration is directly related to the uncertainty in the model parameter vectors. This feature is particularly desirable in nonstationary processes such as grinding-circuit operation since exploration automatically increases

Table 8
Comparing performance metrics for throughput and profit optimisation.

	Throughput	Metal Produced	Electricity Costs	Grinding Costs	Profit
	m ³ /h	g	USD	USD	USD
Throughput	144.3 ± 2.1	34 428 ± 1254	50 287 ± 421	19 100 ± 155	68 326 ± 5288
Profit	105.2 ± 1.6	55 292 ± 702	46 669 ± 236	17 748 ± 86	3 418 992 ± 44 112

when changes occur. However, there is no mechanism to stop the additional exploration towards the end of the experiment in Fig. 8.

One could attempt to curb such exploration for industrial implementation, since the objective function will likely suffer slight decreases during this period. A less aggressive agent with a smaller step size could prevent large movements from the optimum during steadier conditions. However, the unwanted exploration is unlikely to occur in reality. Process systems rarely run at steady-state, as presented in the experiments. Instead, most systems run in a pseudo-steady state with smaller, gradual trends or fluctuations around a mean value. Assuming such movements operate under the same grind curve, the agent's ability to identify the grind curve should not be impaired.

4.4. Convergence times

While these were not explicitly calculated, both optimisers exhibited long convergence times. However, the experiments were initiated at states far from optimal and, coupled with the slow responses of the grinding process, resulted in convergence times upwards of a day. The trajectories observed in the throughput optimisation experiment are more akin to transitioning to a wholly new control philosophy, rather than optimising under an existing philosophy after a disturbance.

Ideally, the CAB agent would converge to the optimal policy between feed ore disturbances. The bulk ore-handling practices of a processing plant strongly affect the frequency of feed-ore disturbances. Plants feeding the mill with run-of-mine ore from a front-end loader will generally experience more variability in the feed than plants that stockpile ore between stages. Furthermore, ore-blending decisions depend on the available ore at the time and can vary daily or even within shifts. Conversely, some plants can receive relatively stable feeds for weeks or months.

Reiterating, the frequency of the feed-ore disturbances is variable and dependent on plant practices. Changes will generally not be as drastic as demonstrated in the experiments here. The agent requires industrial testing to validate whether the system can be optimised or reach the vicinity of the optimum during regular operation.

The CAB agent's exploration mechanism prevented the calculation of reliable convergence times from the time-series data. Continuous exploration prohibited the optimiser from remaining in the 2% convergence band for extended periods. While this instability could be considered a limitation, the agent still produced similar performance, compared to the ESCs in optimising the objective functions. Furthermore, as mentioned in Section 4.3, this exploration is unlikely to encumber the optimiser in an industrial setting where processes run closer to a pseudo-steady state.

5. Conclusion

This paper demonstrated the use of a CAB agent as an optimiser for grinding mills. The testing environment consisted of a dynamic grinding-mill model, calibrated to published grind curve data. The model exhibits steady-state behaviour defined by unknown grind curves that the optimisers had to identify and maximise. The CAB agent's performance was compared to the performance of three ESCs published in the literature.

The agent was tested in throughput and weighted throughput-grind optimisation experiments. In both experiments, the CAB agent performed similarly to the ESCs in maximising the objective function. Both optimisers could steer the process towards the vicinity of the optimum.

Faster convergence times by the CAB agent in the weighted throughput-grind optimisation led to consistently higher average throughputs. Lastly, the agent maximised a comprehensive profit function, thereby demonstrating the agent's ability to learn more complex objective functions in similar time frames.

Sensitivity analyses revealed that the CAB agent proved relatively robust to the two hyper-parameters of the algorithm. Large regions in the search space produced agents with a similar performance. Furthermore, the hyper-parameters are intuitive and can be tuned from process knowledge rather than algorithmic expertise. Further, the employment of linear least-squares algorithms ensures that model training and optimisation processes are on average completed within a fraction of a second, rendering it highly conducive to industrial deployment.

The CAB agent should prove relatively versatile. While the agent was used here to optimise a grinding mill, the optimiser can be applied to any process, as long as individual reward components can be modelled using linear functions. Moreover, the agent's capacity to learn objective functions incorporating numerous key performance indicators and time-varying elements enables the optimisation of an extensive array of processing, economic, or environmental objectives.

Future work involves implementing the CAB agent as a control module in Mintek's advanced process control platform, StarCS, followed by industrial testing on processing plants. Initial tests will be conducted on mills before moving to additional process units, where the shape of possible performance functions is known. Furthermore, extensions of the method can be investigated where the capacity of the underlying linear models can be increased by replacing the polynomial-basis functions with non-linear kernels. Such extensions would likely require more training data, but could still be applicable in cases where the process is generally steadier.

CRedit authorship contribution statement

Jacques Olivier: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis, Data curation. **William John Shipman:** Writing – review & editing, Writing – original draft, Supervision, Software, Project administration, Methodology, Investigation, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

Dr Derick le Roux and Mr Lukasz Ziolkowski, both from the University of Pretoria, kindly provided the MATLAB code for the milling simulator and experimental results used in their publication (Ziolkowski et al., 2022). The MATLAB code was converted to Python and integrated into Mintek's reinforcement learning system.

References

- Abeille, M., 2017. Exploration-Exploitation with Thompson Sampling in Linear Systems (Ph.D. thesis). Université de Lille 1.
- Agarwal, A., Jiang, N., Kakade, S., Sun, W., 2020. Reinforcement learning: Theory and algorithms.
- Agrawal, R., 1995. The continuum-armed bandit problem. *SIAM J. Control Optim.* 33 (6), 1926–1951. <http://dx.doi.org/10.1137/S0363012992237273>.
- Agrawal, S., Goyal, N., 2012a. Analysis of Thompson sampling for the multi-armed bandit problem. In: *Proceedings of the 25th Annual Conference on Learning Theory*. pp. 39.1–39.26.
- Agrawal, S., Goyal, N., 2012b. Thompson sampling for contextual bandits with linear payoffs. *CoRR abs/1209.3352* [arXiv:1209.3352](https://arxiv.org/abs/1209.3352).
- Amin, S., Gomrokchi, M., Satija, H., Van Hoof, H., Precup, D., 2021. A survey of exploration methods in reinforcement learning. [arXiv:2109.00157](https://arxiv.org/abs/2109.00157) [cs].
- Berninger, C., Stöcker, A., Rügamer, D., 2022. A Bayesian time-varying autoregressive model for improved short-term and long-term prediction. *J. Forecast.* 41 (1), 181–200. <http://dx.doi.org/10.1002/for.2802>.
- Bitto, A., Frühwirth-Schnatter, S., 2019. Achieving shrinkage in a time-varying parameter model framework. *J. Econometrics* 210 (1), 75–97. <http://dx.doi.org/10.1016/j.jeconom.2018.11.006>.
- Borell, M., Bäckström, P., Söderberg, L., 1996. Supervisory control of autogenous grinding circuits. *Int. J. Miner. Process.* 44–45, 337–348.
- Bradford, R., Grobler, E., Morgan, P., Vorster, T., 2021. Optimisation of Asanko's Obotan SABC circuit to achieve 'project 5 Million'. In: *Mill Operators Conference 2021*. Brisbane, Qld, Australia, pp. 1–7.
- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., Zaremba, W., 2016. OpenAI gym. [arXiv:1606.01540](https://arxiv.org/abs/1606.01540) [cs].
- Burtini, G., Loepky, J., Lawrence, R., 2015. A survey of online experiment design with the stochastic multi-armed bandit. [arXiv:1510.00757](https://arxiv.org/abs/1510.00757) [cs, stat].
- Cadonna, A., Frühwirth-Schnatter, S., Knaus, P., 2020. Triple the Gamma—A unifying shrinkage prior for variance and variable selection in sparse state space and TVP models. *Econometrics* 8 (2), 20. <http://dx.doi.org/10.3390/econometrics8020020>.
- Cleary, P.W., Owen, P., 2019. Effect of operating condition changes on the collisional environment in a SAG mill. *Miner. Eng.* 132 (September 2018), 297–315. <http://dx.doi.org/10.1016/j.mineng.2018.06.027>, Publisher: Elsevier.
- Coetzee, L.C., Kerrigan, E.C., Craig, I., 2010. Nonlinear model predictive control of a run-of-mine ore milling circuit. *IEEE Trans. Control Syst. Technol.* 18 (1), 222–229. <http://dx.doi.org/10.3182/20080706-5-KR-1001.2587>, ISBN: 9783902661005.
- Craig, I.K., 2012. Grinding mill modeling and control: Past, present and future. In: *Proceedings of the 31st Chinese Control Conference*. Hefei, China, IEEE, pp. 16–21, ISBN: 9789881563811.
- Du Preez, N., Smith, V., Muller, B., 2006. Optimizing the operation of a SAG mill throughput optimum load estimation and control. In: *SAG 2006*. Vancouver, B.C., Canada, pp. 1–13.
- Feng, Z., Liu, G., Wang, L., Gu, Q., Chen, L., 2023. Research on the multiobjective and efficient ore-blending scheduling of open-pit mines based on multiagent deep reinforcement learning. *Sustainability* 15 (6), 5279. <http://dx.doi.org/10.3390/su15065279>.
- Góralczyk, M., Krot, P., Zimroz, R., Ogonowski, S., 2020. Increasing energy efficiency and productivity of the comminution process in tumbling mills by indirect measurements of internal dynamics—an overview. *Energies* 13 (24), <http://dx.doi.org/10.3390/en13246735>.
- Guo, L., Wang, H., Zhang, J., 2019. Data-driven grinding control using reinforcement learning. In: *2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems*. HPCC/SmartCity/DSS, Zhangjiajie, China, ISBN: 978-1-72812-058-4, pp. 2817–2824. <http://dx.doi.org/10.1109/HPCC/SmartCity/DSS.2019.00395>.
- Hadizadeh, M., Farzanegan, A., Noaparast, M., 2017. Supervisory fuzzy expert controller for sag mill grinding circuits: Sungun copper concentrator. *Min. Process. Extract. Metall. Rev.* 38 (3), 168–179. <http://dx.doi.org/10.1080/08827508.2017.1281133>, Publisher: Taylor & Francis.
- Hallen, M., Astrand, M., Sikstrom, J., Servin, M., 2019. Reinforcement learning for grinding circuit control in mineral processing. In: *2019 24th IEEE International Conference on Emerging Technologies and Factory Automation*. ETFA, Zaragoza, Spain, ISBN: 978-1-72810-303-7, pp. 488–495. <http://dx.doi.org/10.1109/ETFA.2019.8869212>.
- Hilden, M.M., Powell, M.S., Yahyai, M., 2021. An improved method for grinding mill filling measurement and the estimation of load volume and mass. *Miner. Eng.* 160 (September 2020), <http://dx.doi.org/10.1016/j.mineng.2020.106638>, Publisher: Elsevier.
- Huber, P., 1964. Robust estimation of a location parameter. *Ann. Math. Stat.* 35 (1), 73–101.
- Huber, P., 1981. *Robust Statistics*. New York.
- Huo, D., Sari, Y.A., Kealey, R., Zhang, Q., 2023. Reinforcement learning-based fleet dispatching for greenhouse gas emission reduction in open-pit mining operations. *Resour. Conserv. Recy.* 188, 106664. <http://dx.doi.org/10.1016/j.resconrec.2022.106664>.
- Knaus, P., Bitto-Nemling, A., Cadonna, A., Frühwirth-Schnatter, S., 2021. Shrinkage in the time-varying parameter model framework using the R package shrinkTVP. *J. Stat. Softw.* 100, 1–32. <http://dx.doi.org/10.18637/jss.v100.i13>.
- Kraft, D., 1988. A Software Package for Sequential Quadratic Programming. Technical Report, DFVLR Institut für Dynamik der Flugsysteme, Oberpfaffenhofen, Bavaria, Germany.
- Krstic, M., Wang, H., 2000. Stability of extremum seeking feedback for general nonlinear dynamic systems. *Automatica* 36.
- Kumar, A., Dimitrakopoulos, R., Maulen, M., 2020. Adaptive self-learning mechanisms for updating short-term production decisions in an industrial mining complex. *J. Intell. Manuf.* 31 (7), 1795–1811. <http://dx.doi.org/10.1007/s10845-020-01562-5>.
- Lattimore, T., Szepesvári, C., 2020. *Bandit Algorithms*. Cambridge.
- Le Roux, J.D., Craig, I.K., Hulbert, D.G., Hinde, A.L., 2013. Analysis and validation of a run-of-mine ore grinding mill circuit model for process control. *Miner. Eng.* 43–44, 121–134. <http://dx.doi.org/10.1016/j.mineng.2012.10.009>.
- Le Roux, J., Olivier, L., Naidoo, M., Padhi, R., Craig, I., 2016. Throughput and product quality control for a grinding mill circuit using non-linear MPC. *J. Process Control* 42, 35–50. <http://dx.doi.org/10.1016/j.jprocont.2016.04.007>.
- Le Roux, J.D., Steinboeck, A., Kugi, A., Craig, I.K., 2020. Steady-state and dynamic simulation of a grinding mill using grind curves. *Miner. Eng.* 152, 106208. <http://dx.doi.org/10.1016/j.mineng.2020.106208>.
- Levinson, Z., Dimitrakopoulos, R., Keutchan, J., 2023. Simultaneous stochastic optimization of an open-pit mining complex with preconcentration using reinforcement learning. *Appl. Soft Comput.* 138, 110180. <http://dx.doi.org/10.1016/j.asoc.2023.110180>.
- Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D., 2016. Continuous control with deep reinforcement learning. In: *Proceedings of the 4th International Conference on Learning Representations*. San Juan, Puerto Rico, [arXiv:1509.02971](https://arxiv.org/abs/1509.02971) [cs, stat].
- McBride, A., Powell, M., 2006. A structured approach to modelling SAG mill liner wear - Numerical modelling of liner evolution. In: *SAG 2006*, Vol. III. Vancouver, B.C., Canada, pp. 120–132.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., Hassabis, D., 2015. Human-level control through deep reinforcement learning. *Nature* 518 (7540), 529–533. <http://dx.doi.org/10.1038/nature14236>.
- Morrell, S., 2003. *Grinding Mills : How to Accurately Predict Their Power Draw*. Cape Town, South Africa, ISBN: 0958460922.
- Napier-Munn, T., 2015. Is progress in energy-efficient comminution doomed? *Miner. Eng.* 73, 1–6. <http://dx.doi.org/10.1016/j.mineng.2014.06.009>, Publisher: Elsevier Ltd.
- Nian, R., Liu, J., Huang, B., 2020. A review on reinforcement learning: Introduction and applications in industrial process control. *Comput. Chem. Eng.* 139, 106886. <http://dx.doi.org/10.1016/j.compchemeng.2020.106886>.
- Olivier, L.E., Craig, I.K., 2017. A survey on the degree of automation in the mineral processing industry. In: *2017 IEEE AFRICON: Science, Technology and Innovation for Africa*. Cape Town, South Africa, ISBN: 978-1-5386-2775-4, pp. 404–409. <http://dx.doi.org/10.1109/AFRCON.2017.8095516>.
- Owen, P., Cleary, P.W., 2015. The relationship between charge shape characteristics and fill level and lifter height for a SAG mill. *Miner. Eng.* 83, 19–32. <http://dx.doi.org/10.1016/j.mineng.2015.08.009>, Publisher: Elsevier Ltd.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E., 2011. Scikit-learn: Machine Learning in Python. *J. Mach. Learn. Res.* 12, 2825–2830.
- Powell, M., Mainza, A., 2006. Extended grinding curves are essential to the comparison of milling performance. *Miner. Eng.* 19 (15), 1487–1494. <http://dx.doi.org/10.1016/j.mineng.2006.08.004>.
- Powell, M.S., Van der Westhuizen, A.P., Mainza, A.N., 2009. Applying grindcurves to mill operation and optimisation. *Miner. Eng.* 22 (7–8), 625–632. <http://dx.doi.org/10.1016/j.mineng.2009.01.008>, Publisher: Elsevier Ltd.
- Radziszewski, P., Martins, S., 2019. Contributing to a better understanding of grinding media wear - Exploring the impact of different parameters. In: *SAG Conference 2019*. Vancouver, B.C., Canada, pp. 1–21.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. [arXiv:1707.06347](https://arxiv.org/abs/1707.06347) [cs].
- Seabold, S., Perktold, J., 2010. Statsmodels: Econometric and statistical modelling with python. In: *9th Python in Science Conference*.
- Shipman, W.J., 2021. Optimizing milling operations using continuum-armed bandits with multiple reward components and constraints. In: *Conference Proceedings: the 2nd Southern African Conference for Artificial Intelligence Research*. Online, p. 15.
- Sutton, R.S., Barto, A., 2018. *Reinforcement Learning: An Introduction*, second ed. Adaptive Computation and Machine Learning, Cambridge, Massachusetts, London, England, ISBN: 978-0-262-03924-6.
- Thivierge, A., Bouchard, J., Desbiens, A., Pérez-G., E.M., 2019. Modeling the product net value of a grinding-flotation circuit. *IFAC-PapersOnLine* 52 (14), 18–23. <http://dx.doi.org/10.1016/j.ifacol.2019.09.157>.
- Thompson, W., 1933. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika* 25, 285–294. <http://dx.doi.org/10.1093/biomet/25.3-4.285>.

- Virtanen, P., Gommers, R., Oliphant, T.E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S.J., Brett, M., Wilson, J., Millman, K.J., Mayorov, N., Nelson, A.R.J., Jones, E., Kern, R., Larson, E., Carey, C.J., Polat, I., Feng, Y., Moore, E.W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E.A., Harris, C.R., Archibald, A.M., Ribeiro, A.H., Pedregosa, F., van Mulbregt, P., SciPy 1.0 Contributors, 2020. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nat. Methods* 17, 261–272. <http://dx.doi.org/10.1038/s41592-019-0686-2>.
- Wei, D., Craig, I.K., 2009. Economic performance assessment of two ROM ore milling circuit controllers. *Miner. Eng.* 22 (9–10), 826–839. <http://dx.doi.org/10.1016/j.mineng.2009.02.013>, Publisher: Elsevier Ltd.
- Van der Westhuizen, A., Powell, M.S., 2006. Milling curves as a tool for characterising SAG mill performance. In: *SAG 2006*, Vol. I. Vancouver, B.C., Canada, pp. 217–232.
- Wills, B., Napier-Munn, T., 2006. Wills' mineral processing technology. ISBN: 0-7506-4450-8, <http://dx.doi.org/10.1016/b978-075064450-1/50000-x>.
- Ziolkowski, L., Le Roux, J., Craig, I., 2021. Optimizing grinding mill performance using extremum seeking control. *IFAC-PapersOnLine* 54 (11), 43–48. <http://dx.doi.org/10.1016/j.ifacol.2021.10.048>.
- Ziolkowski, L., Le Roux, J., Craig, I., 2022. Extremum seeking control for optimization of an open-loop grinding mill using grind curves. *J. Process Control* 114, 54–70. <http://dx.doi.org/10.1016/j.jprocont.2022.04.005>.